



Mahidol University

Wisdom of the Land

ITCS215
Introduction to Interactive Multimedia Technology

Class: 1-2026
Module III: Introduction to Computer Graphics, Generative AI, and Audio
Lecture: TUE 13:00-16:00

Unit 3: 3D Graphics, Motion, and Audio (Voice AI)

Instructor: Lecturer Dr. Kanoksak Wattanachote, PhD (金功勳博士)
Contact: kanoksak.wat@mahidol.ac.th

Grading

ITCS215 Introduction to Interactive Multimedia Technology

- Overall Score Parts
  - Class activities + Assignment 30%
  - Quizzes 30%
  - Final Exam 40%
- Module 1 (5 weeks): Introduction to Animation, Interactive Application, Text, and Font.
- Module 2 (5 weeks)**:** Introduction to Image Analysis and Machine Learning.
- **Module 3** (5 weeks): **Introduction to Computer Graphics, Generative AI, Sound-Visualization/Analytics**.
  - Class activities + Assignment 10%
  - Quizzes 10%
  - Final Exam 40/3 = 13.33%

Unit 3: Content

Unit 3: 3D Graphics, Motion, and Audio (Voice AI)

- 3D Computer Graphics
- Animation Concepts (3D Animation) cont’
- Motion Analysis
- Voice AI Creation

Introduction
to

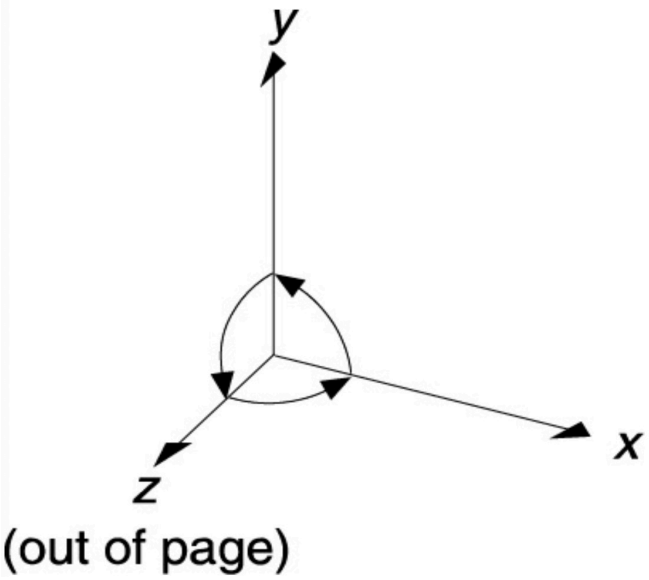
3D Computer Graphics



Representation of 3D

Module III: 3D Computer Graphics

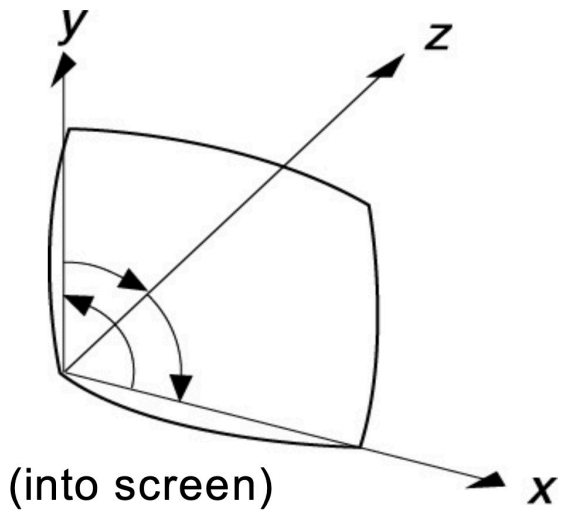
- Z axis represents depth
- Right-Handed System
  - When looking “down” at the origin, positive rotation is CCW.



Representation of 3D

Module III: 3D Computer Graphics

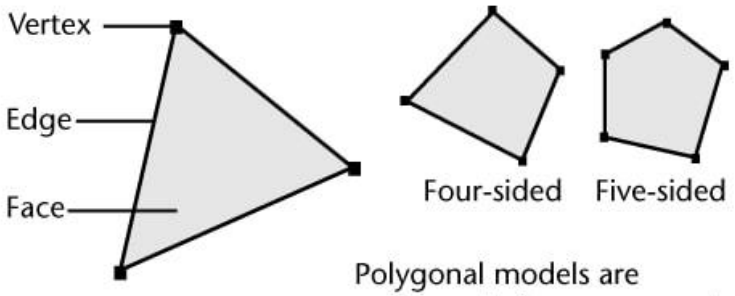
- Left-Handed System
- When looking “down”, positive rotation is in CW
  - More natural interpretation for displays,
  - Big Z means “far”.



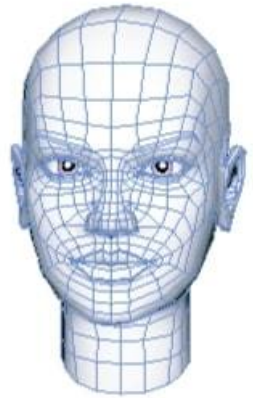
3D Object

Module III: 3D Computer Graphics

- At a fundamental level, you'll need to know the building blocks of 3D modeling.
- Here are the most important terms:
  - Vertex: A single point and the smallest component of a 3D model.
  - Edge: A straight line that connects two vertices.
  - Edges define the shape of a 3D model.
  - Polygon: Any shape that's formed by connecting straight lines.
    - There are several types of polygons (equilateral, equiangular, regular, irregular, cyclic, convex, concave) depending on the number of sides and the extent of the angles.



Polygonal models are composed of many separate polygons combined into a polygon mesh.

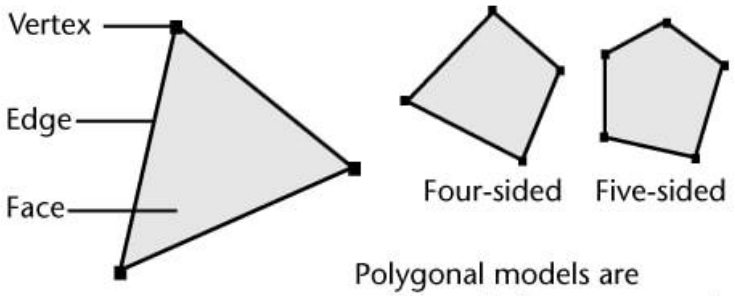


source: https://all3dp.com/2/3d-modeling-basics-simply-explained/

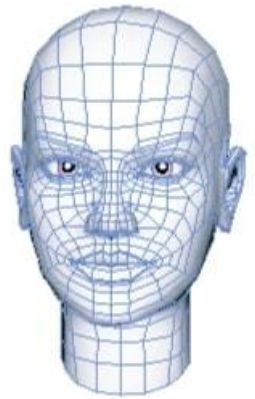
3D Object

Module III: 3D Computer Graphics

- At a fundamental level, you'll need to know the building blocks of 3D modeling.
- Here are the most important terms:
  - Face: The most basic part of a polygon mesh.
    - It is the space between the edges
  - Mesh: A collection of polygons that are connected along their edges.
    - A 3D object consists of one or more 3D meshes.



Polygonal models are composed of many separate polygons combined into a polygon mesh.

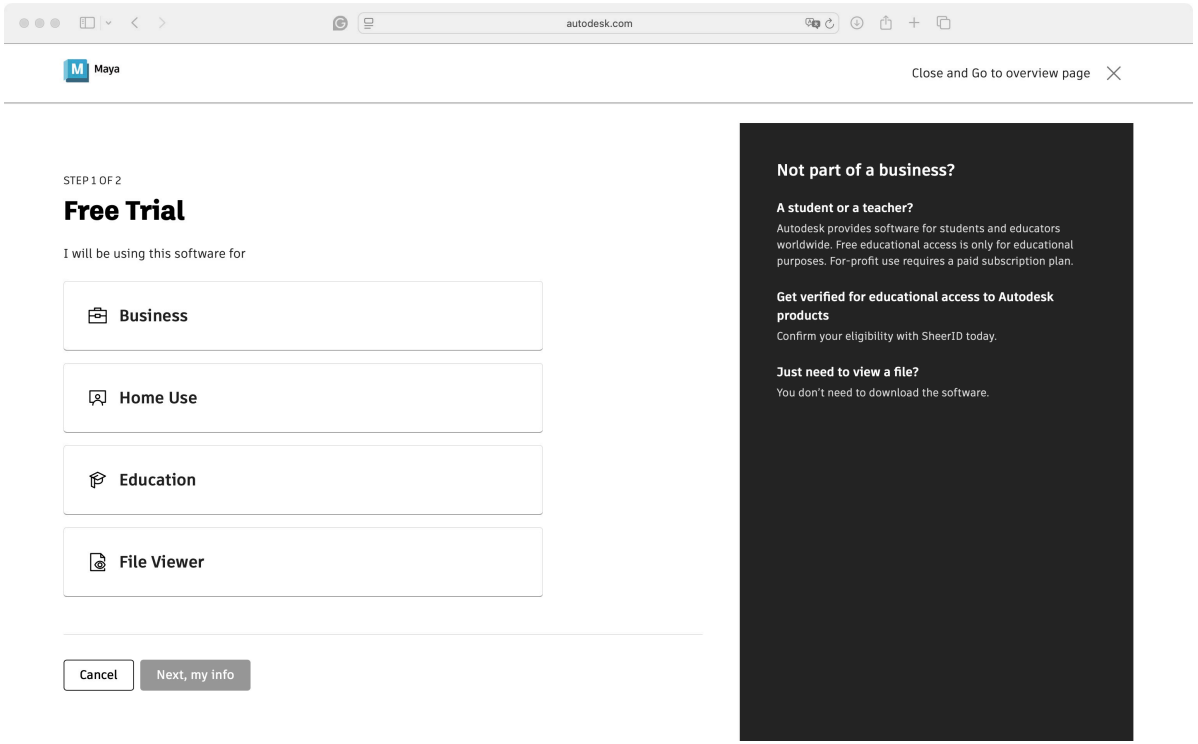


source: https://all3dp.com/2/3d-modeling-basics-simply-explained/

3D Graphics Tools

Module III: Maya

- Autodesk Maya is professional 3D software for creating realistic characters and blockbuster-worthy effects.
- Bring believable characters to life with engaging animation tools.
  Shape 3D objects and scenes with intuitive modeling tools.
  Create realistic effects—from explosions to cloth simulation.

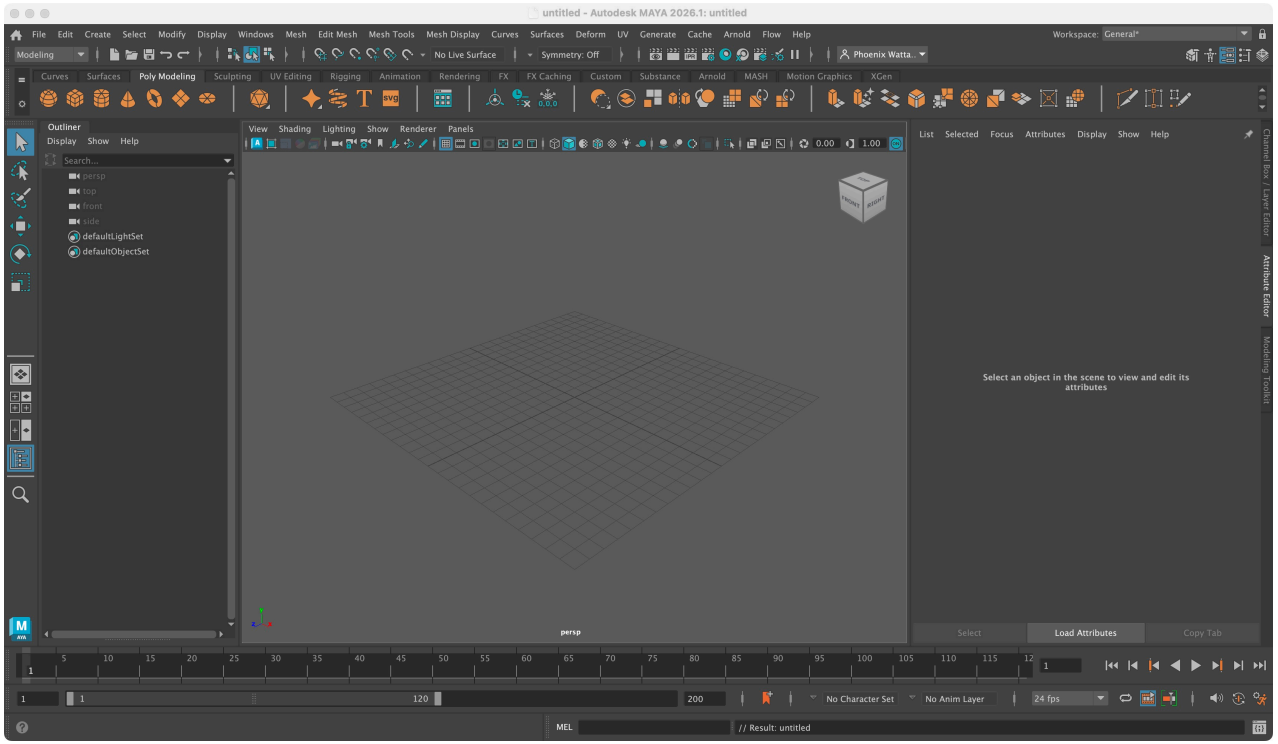


source: https://www.autodesk.com/asean/products/maya/trial-intake

3D Graphics Tools

Module III: Maya

- Every 3D modeling program uses a 3D environment, which is made navigable using vectors and planes.
  - Plane: A two-dimensional surface oriented along one of the 3 axes.

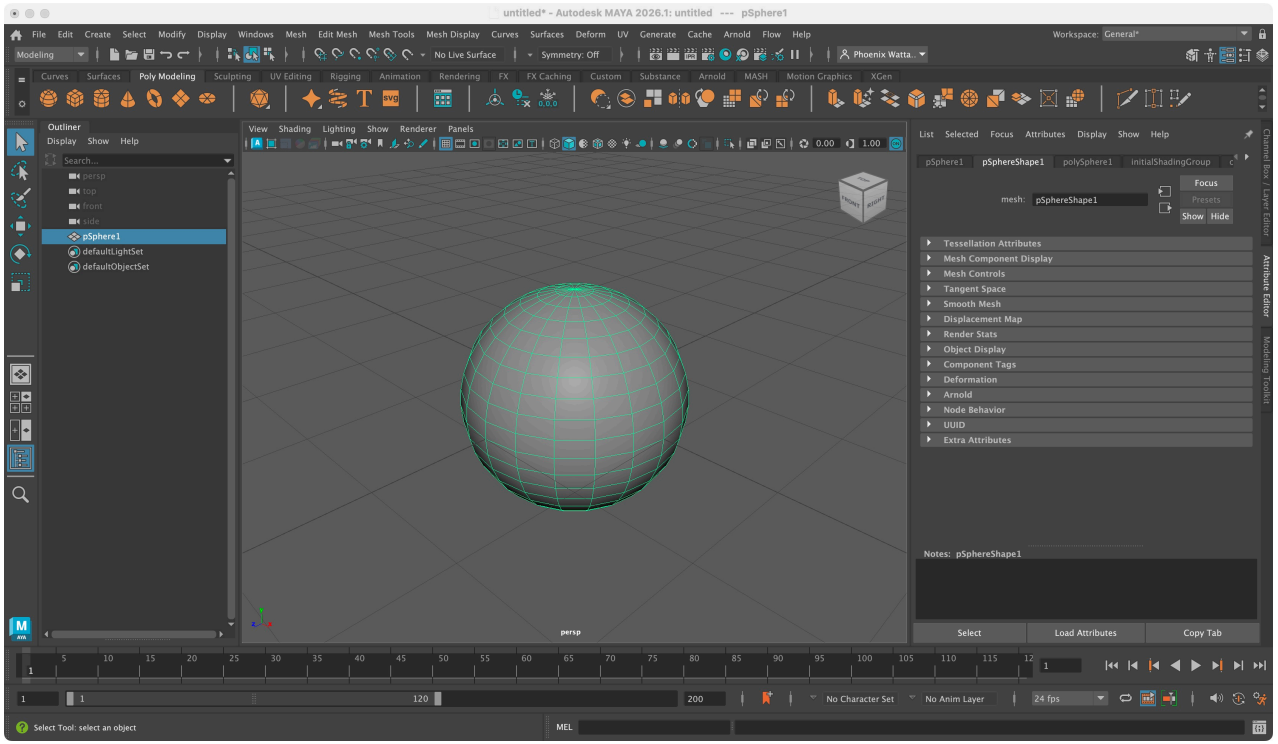


source: https://www.autodesk.com/asean/products/maya/trial-intake

3D Graphics Tools

Module III: Maya

- Every 3D modeling program uses a 3D environment, which is made navigable using vectors and planes.
  - Plane: A two-dimensional surface oriented along one of the 3 axes.
  - Vector: Indicates directionality and orientation, usually along the 3 axes.

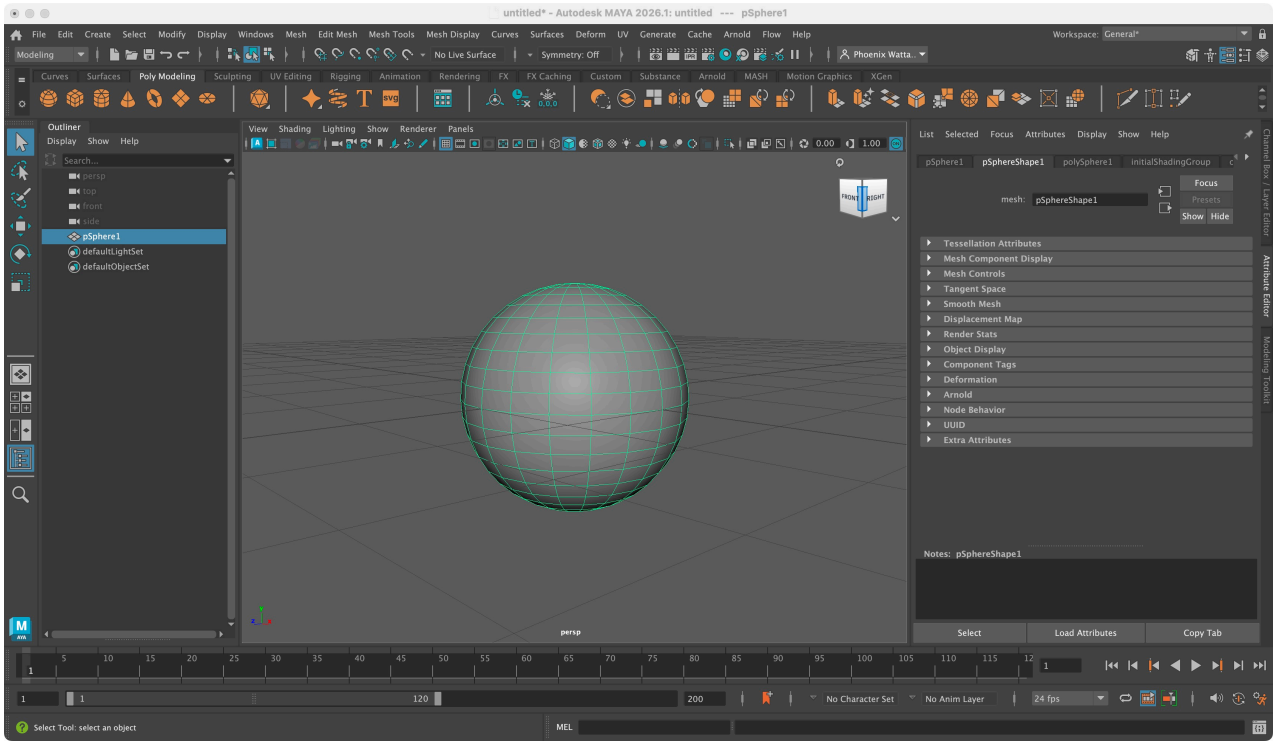


source: https://www.autodesk.com/asean/products/maya/trial-intake

3D Graphics Tools

Module III: Maya

- Every 3D modeling program uses a 3D environment, which is made navigable using vectors and planes.
  - Plane: A two-dimensional surface oriented along one of the 3 axes.
  - Vector: Indicates directionality and orientation, usually along the 3 axes.

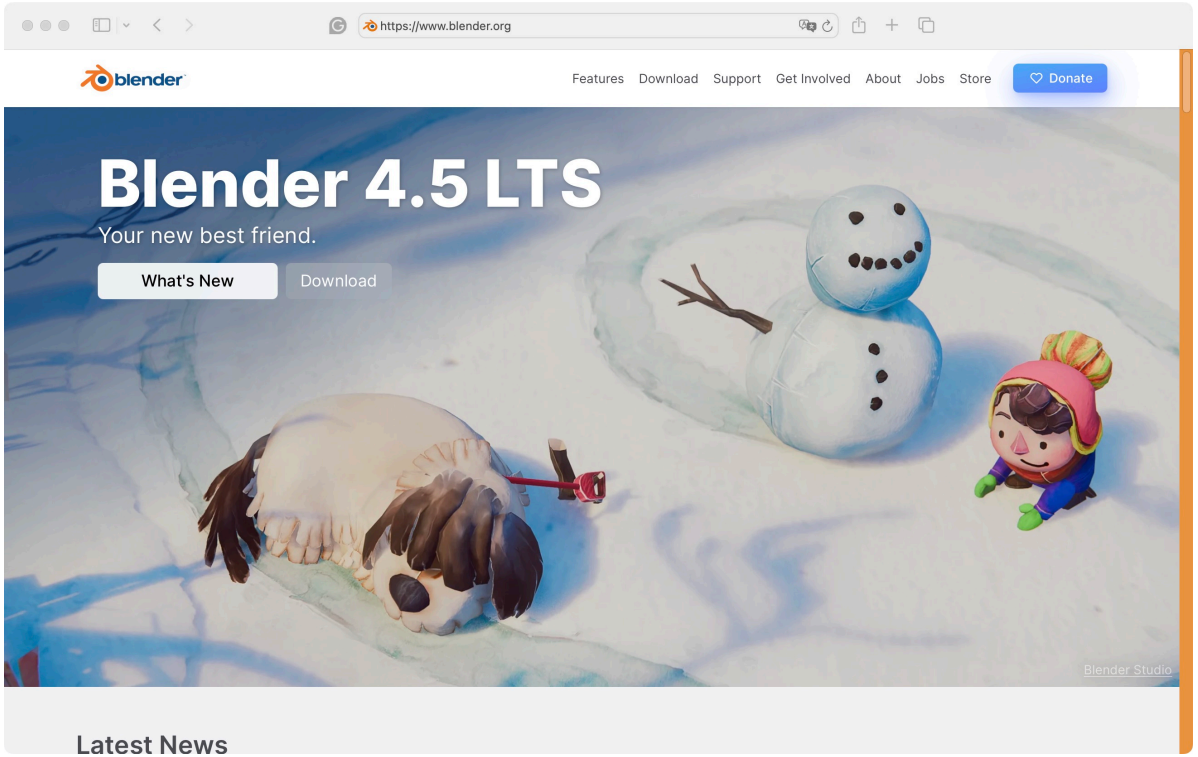


source: https://www.autodesk.com/asean/products/maya/trial-intake

3D Graphics Tools

Module III: Blender

- Blender is a free and open source 3D creation suite.
- It supports the entirety of the 3D pipeline—modeling, rigging, animation, simulation, rendering, compositing, and motion tracking, even video editing and game asset creation.
- Blender is cross-platform and runs equally well on Linux, Windows, and Macintosh computers.

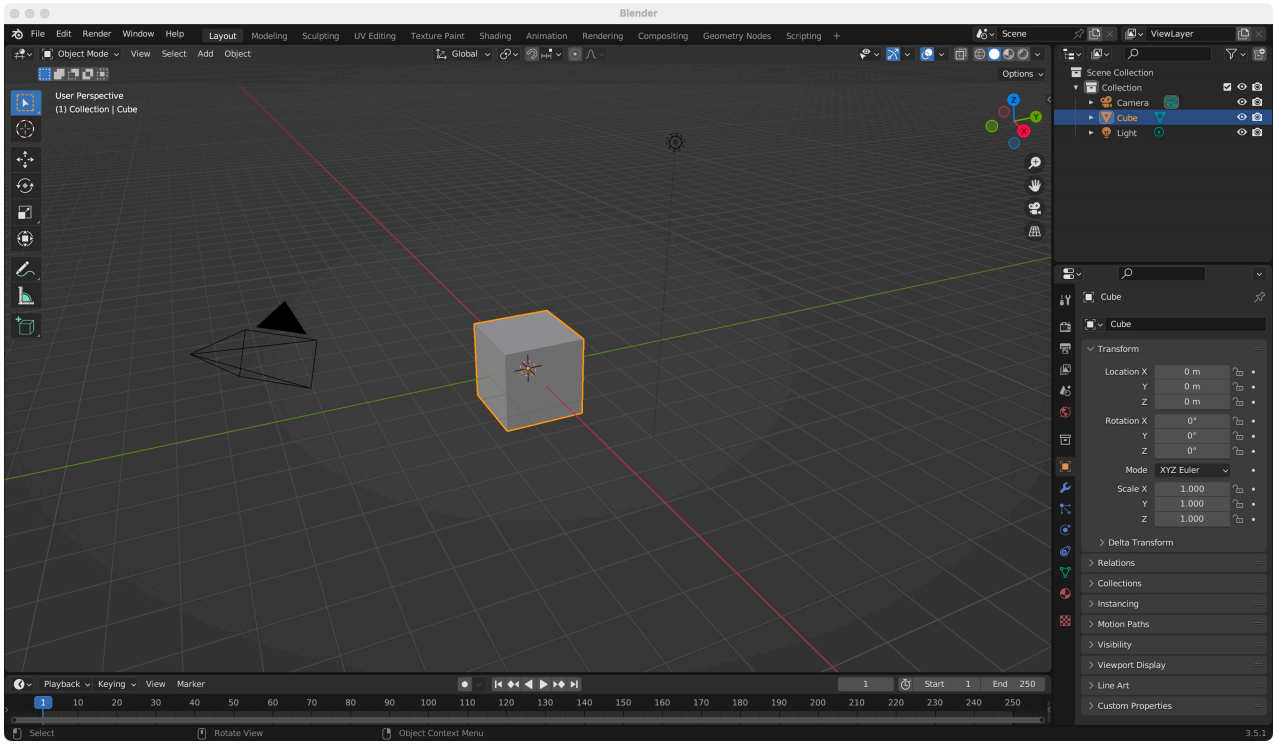


source: https://www.blender.org

3D Graphics Tools

Module III: Blender

- Blender also uses a 3D environment, which is navigable using vectors and planes.
  - Plane: A two-dimensional surface oriented along one of the 3 axes.
  - Vector: Indicates directionality and orientation, usually along the 3 axes.

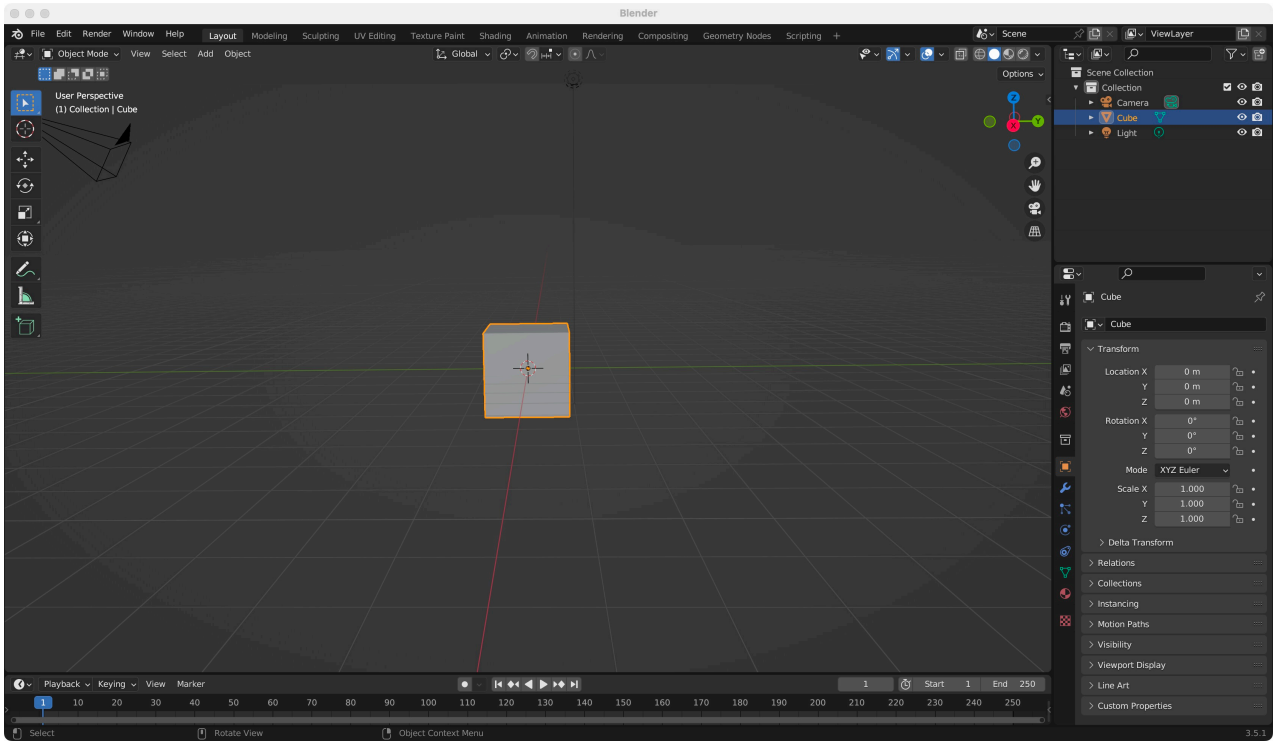


source: https://www.blender.org

3D Graphics Tools

Module III: Blender

- Blender also uses a 3D environment, which is navigable using vectors and planes.
  - Plane: A two-dimensional surface oriented along one of the 3 axes.
  - Vector: Indicates directionality and orientation, usually along the 3 axes.



source: https://www.blender.org

3D Modeling/Polygon Modeling

Module III: 3D Computer Graphics

- Polygon modeling is the foundational technique for constructing 3D models in digital environments.
- It is akin to "digital sculpting with geometric building blocks."
- It involves creating a model's surface by connecting and manipulating individual flat, geometric shapes—polygons—to form a complex whole called a mesh.
- Polygon modeling is the foundational technique for constructing 3D models in digital environments.





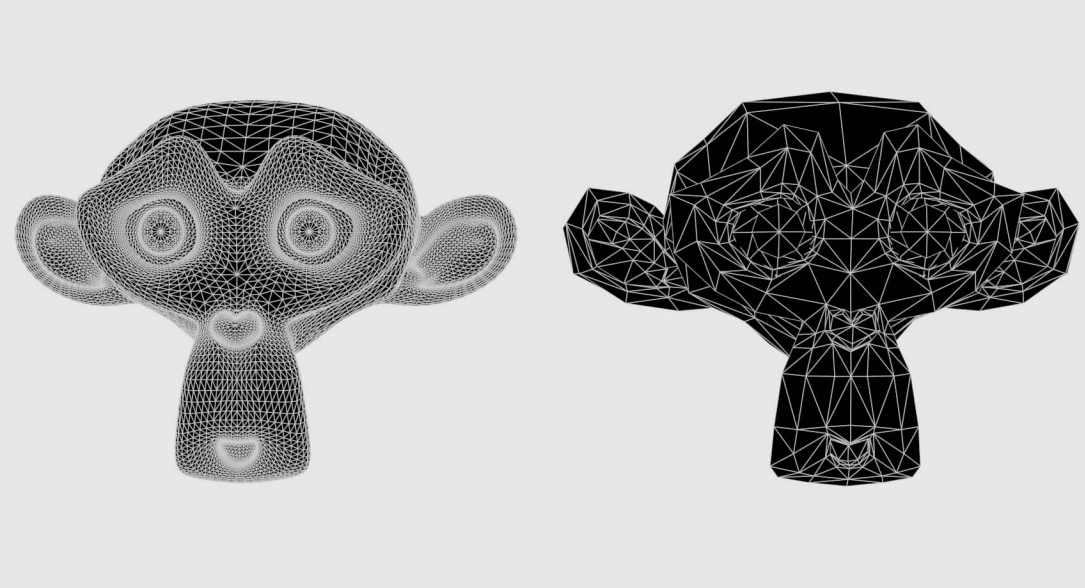
source: https://3dstudio.co/low-and-high-poly-modeling/

3D Modeling/Polygon Modeling

Module III: 3D Computer Graphics

- The Core Components (Breaking Down the Mesh)
- A polygon mesh is not a single, solid object.
- It's a collection of interconnected elements.

- The interconnected elements:
  - Vertices (singular: Vertex): These are the most basic elements—points or dots in 3D space defined by their X, Y, and Z coordinates.
    - They are the “corners” of your polygons.

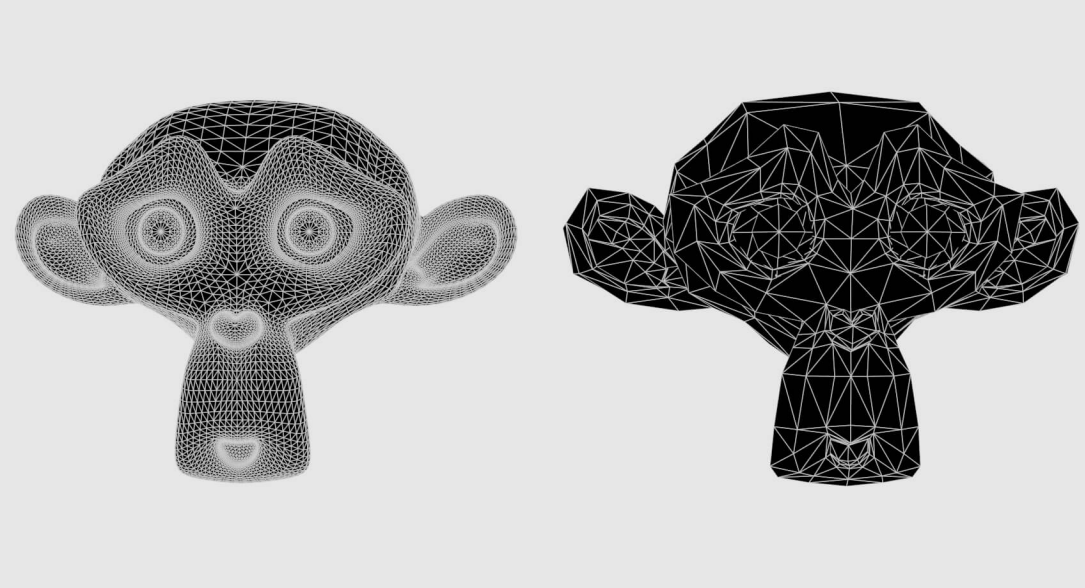


source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

3D Modeling/Polygon Modeling

Module III: 3D Computer Graphics

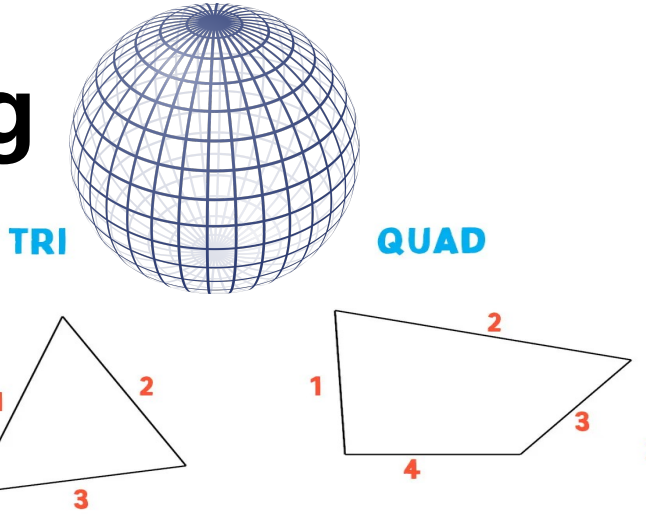
- Edges: These are the straight lines that connect two vertices.
  - They form the border of a polygon.
- Faces: This is the actual polygon itself.
  - It is the closed, flat, 2D shape formed when three or more edges form a connected loop.



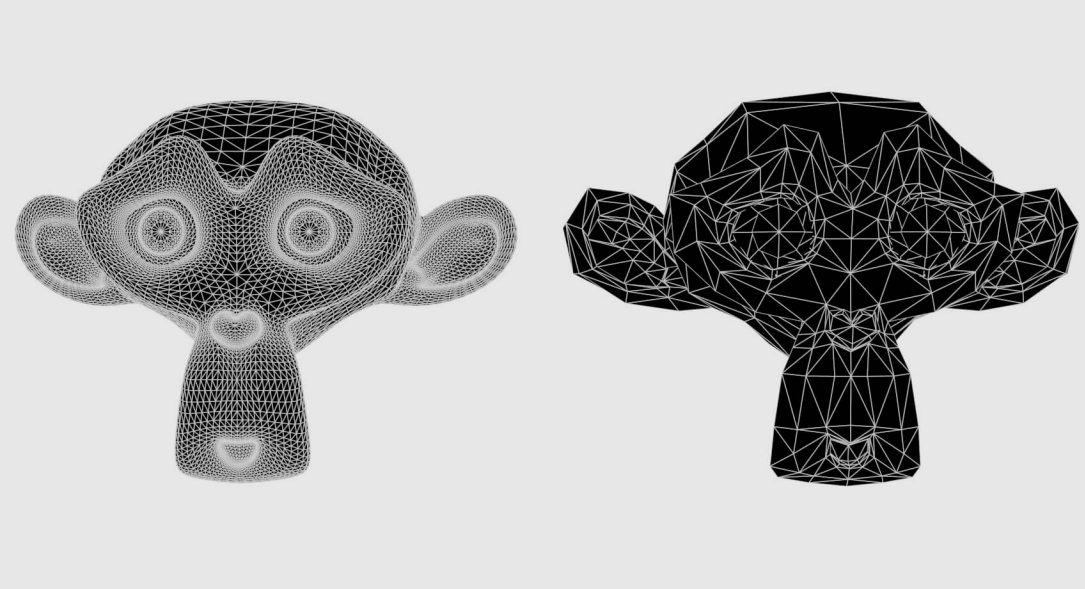
source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

3D Modeling/Polygon Modeling

Module III: 3D Computer Graphics



- Faces: The most common faces are:
  - Tris (Triangles): Faces with 3 sides.
    - They are the simplest and most stable polygons.
    - All real-time graphics (games) ultimately convert models to triangles for rendering.
  - Quads (Quadrilaterals): Faces with 4 sides.
    - These are the preferred choice for most artists during the modeling process because they subdivide smoothly and are easier to deform and sculpt.

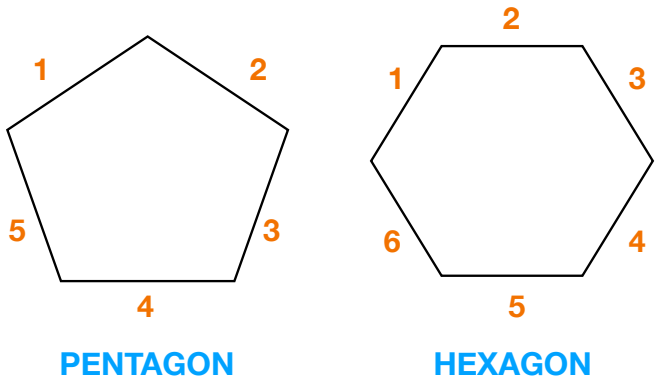


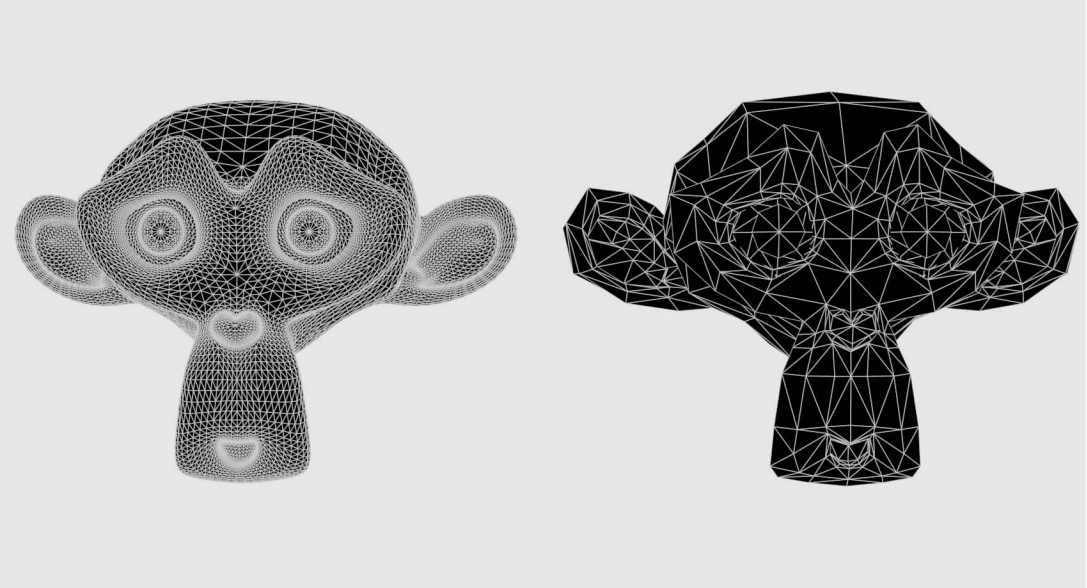
source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

3D Modeling/Polygon Modeling

Module III: 3D Computer Graphics

- Faces: The most common faces are:
  - N-Gons: Faces with 5 or more sides (e.g., pentagons, hexagons).
    - While sometimes useful, they are generally avoided as they can cause rendering errors, odd shading, and problems during subdivision.



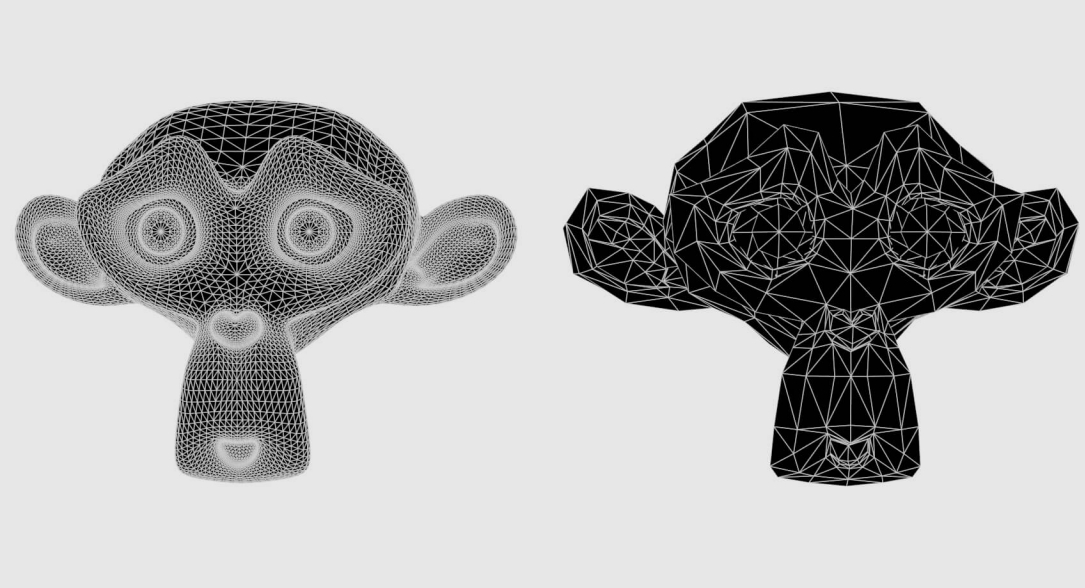


source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

Low-polygon Model

Module III: 3D Computer Graphics

- Low-polygon models will come in handy when you develop a game and need to do a lot of on-the-fly rendering, and are called “low-poly”
- They use less computational power.
  - So they render extremely fast compared to high-polygon models that take hours to complete.
  - Low poly refers to using fewer polygons, fewer than 100,000, to create a polygonal mesh.
  - A low-poly 3D model is a 3D asset with a comparatively lower poly count than high-poly models and is easy to edit, load, and view because it is lightweight.

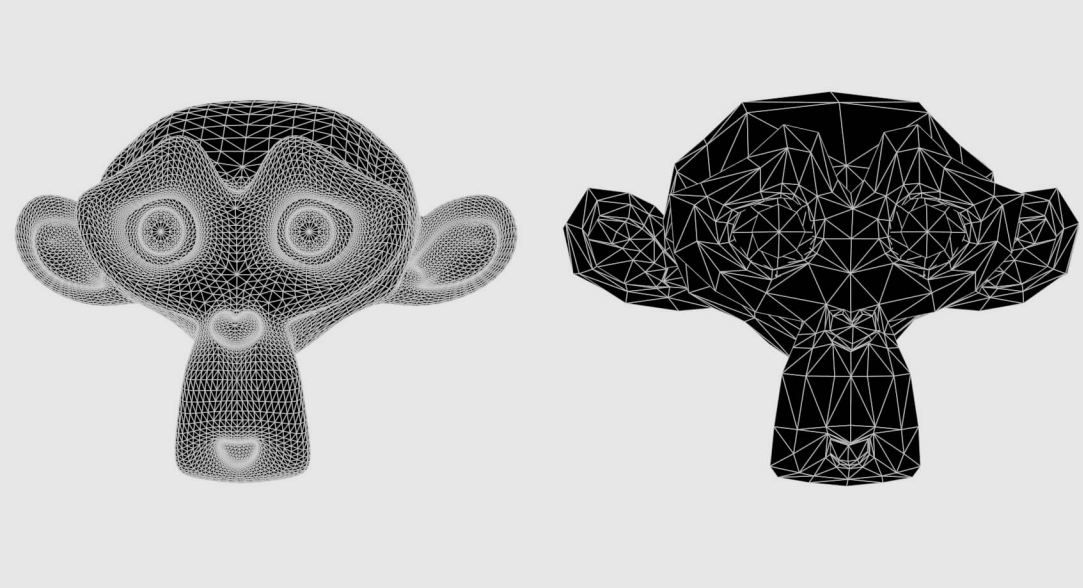


source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

Low-polygon Model

Module III: 3D Computer Graphics

- Low-poly 3D models have simple geometric shapes and are perfect for real-time renderings such as computer games.
- Low-poly models lack super-realistic details and are used for visualizations that do not require such great details.

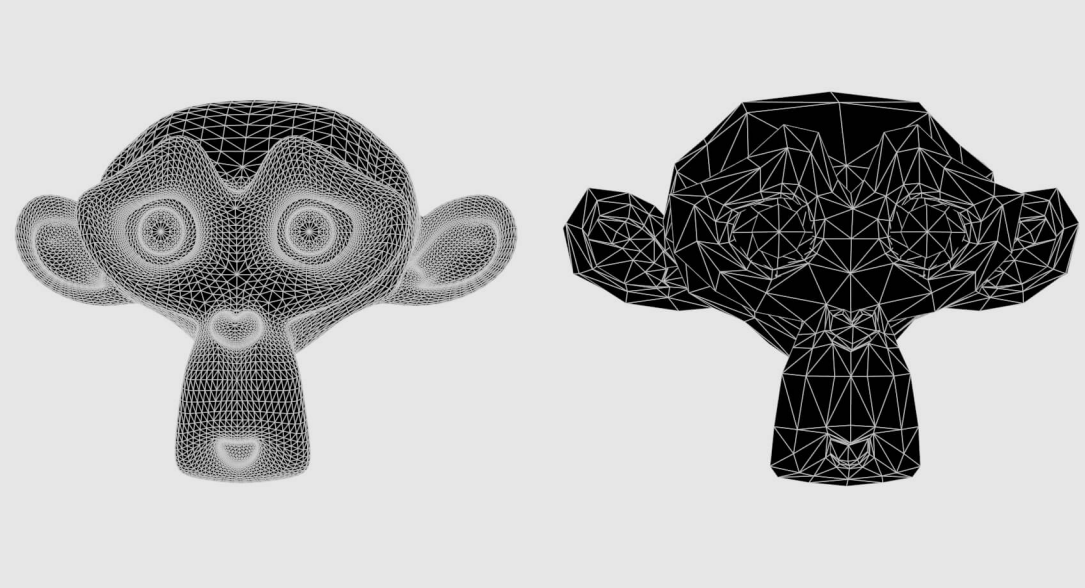


source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

High-polygon Model

Module III: 3D Computer Graphics

- The main thing that helps you differentiate between low and high polygon models is the level of detail.
- High-polygon models are more detailed and can be called “high-poly”.
- Though a high number of polygons allows you to achieve finer details.
- High-polygon models are hard to work with in terms of loading, viewing, and editing.
- It takes time to load the edits and move around the viewpoint.
- So, high-polygon modeling is considered “heavier”.

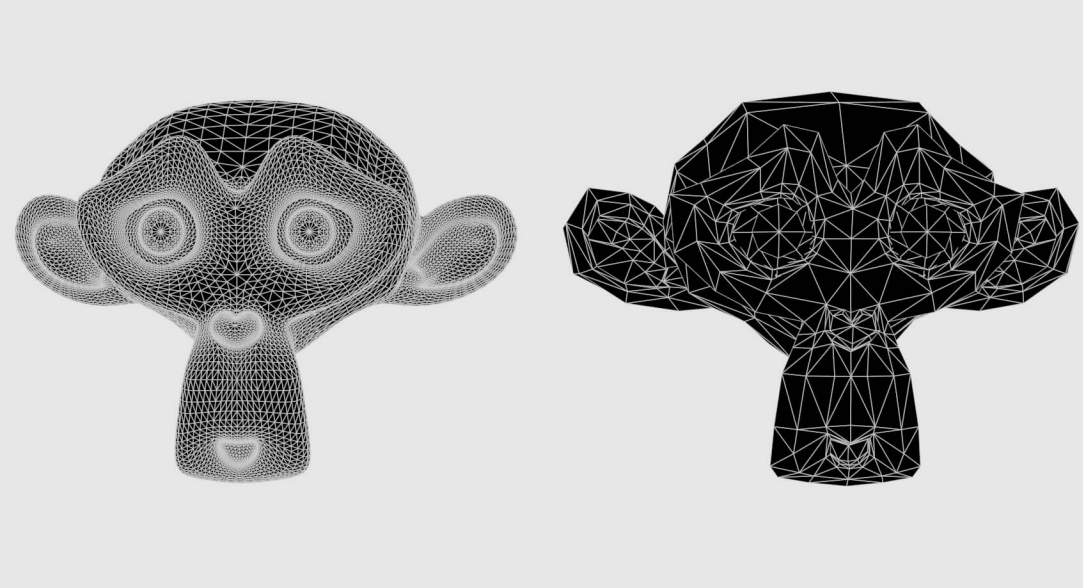


source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

High-polygon Model

Module III: 3D Computer Graphics

- "High poly" refers to the usage of more than 100,000 polygons based on the type, size, and complexity level of the model to create a mesh.
- High poly 3D models are hyper-detailed and ultra-realistic 3D replicas of real-world objects.
- Highly poly means having a high polygon count and complex geometry, which makes a 3D model so realistic.



source: https://www.modelry.ai/blog/difference-between-high-poly-and-low-poly-models

Low-polygon vs. High-polygon Model

Module III: 3D Computer Graphics

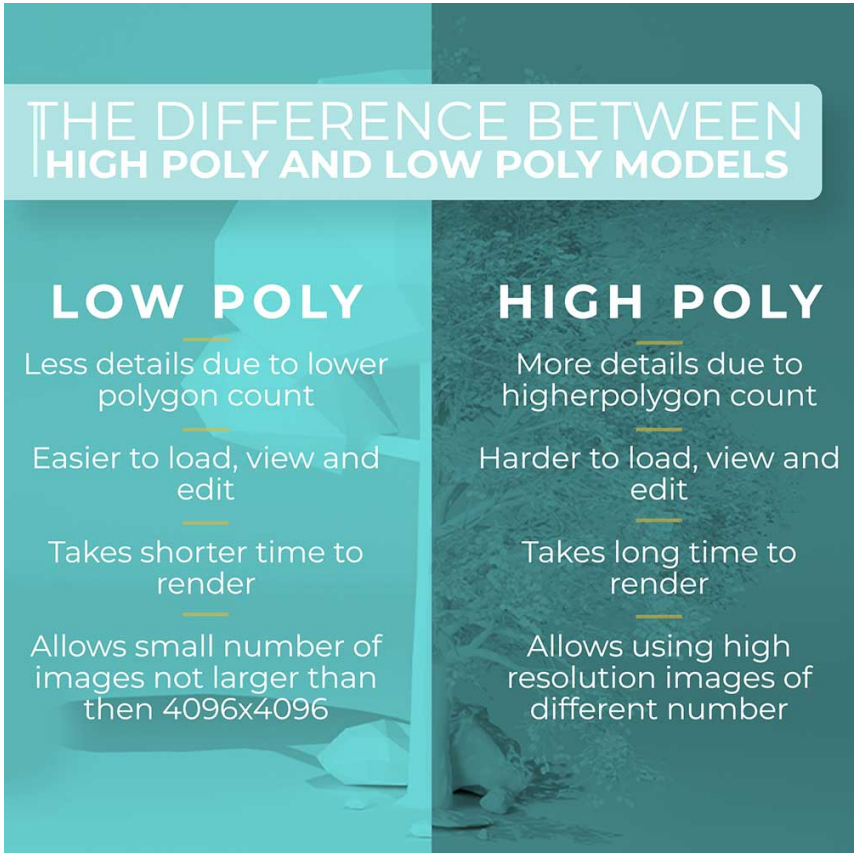


source: https://professional3dservices.com/blog/high-poly-vs-low-poly-3d-models-difference.html

- Which one should we choose? When?







source: https://3dstudio.co/low-and-high-poly-modeling/

Applications of Low-poly Models

Module III: 3D Computer Graphics

- Low-poly modeling is a technique that uses a minimum number of polygons to create a simplified 3D model, often used for video game applications.
- Low-poly modeling should be your option if visual detail is not your priority and if you need your product demonstrations to be interactive.



source: https://professional3dservices.com/blog/high-poly-vs-low-poly-3d-models-difference.html

Applications of Low-poly Models

Module III: 3D Computer Graphics

- Here are the scenarios where low-poly modeling is used:
  - Low-poly models are used in 3D viewers that facilitate real-time model calibration.
  - e-Commerce businesses use low-poly 3D viewers for product demonstrations.
  - Prospective customers can rotate these product images and check out environments where detail is not necessary.
  - Low-poly models find their use in VR applications.
  - Using low-poly models, you can design interactive environments.



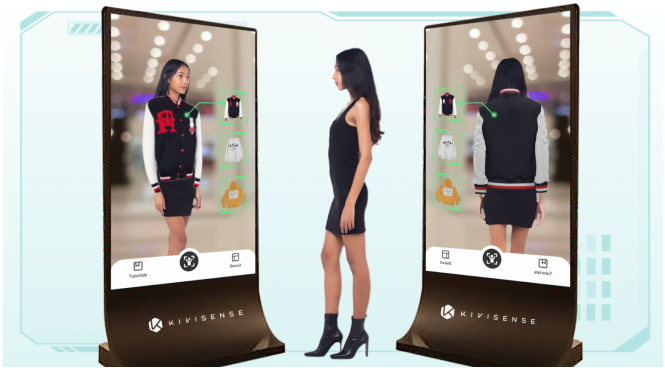


source: https://www.livehome3d.com/fields-of-use/vr-interior-design-app

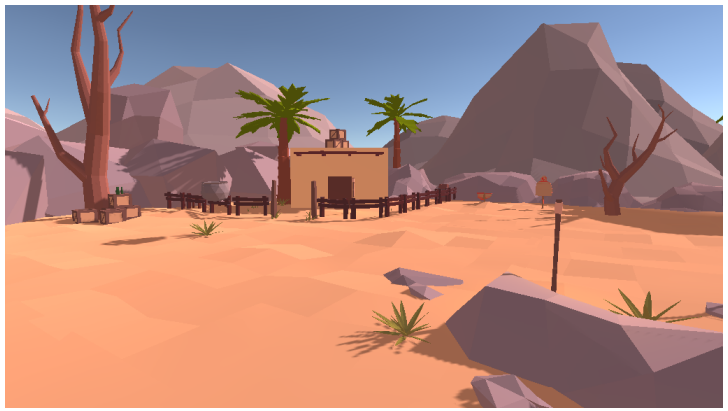
Applications of Low-poly Models

Module III: 3D Computer Graphics

- Here are the scenarios where low-poly modeling is used:
  - Use of AR applications that allow customers to judge the efficacy of products in a real setting, we should opt for low-poly models.
  - Low-poly models find their widespread use in 3D gaming.
  - Low-poly models applied to the intelligent vehicles.

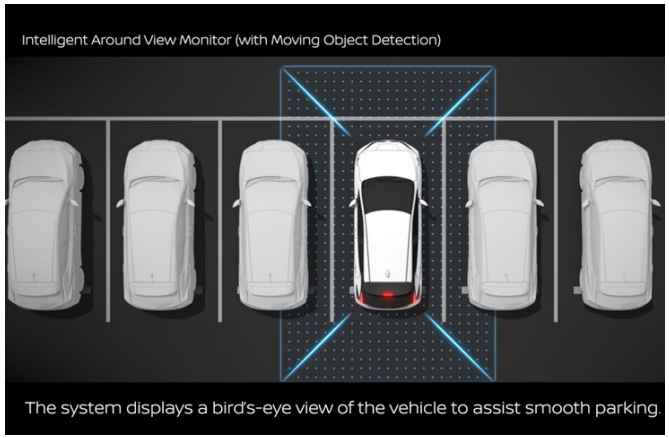


source: https://tryon.kivisense.com/blog/augmented-reality-in-clothing/



source: https://buggame.itch.io/low-poly-game-world-walkthrough/devlog/37648/vr-low-poly-game-world



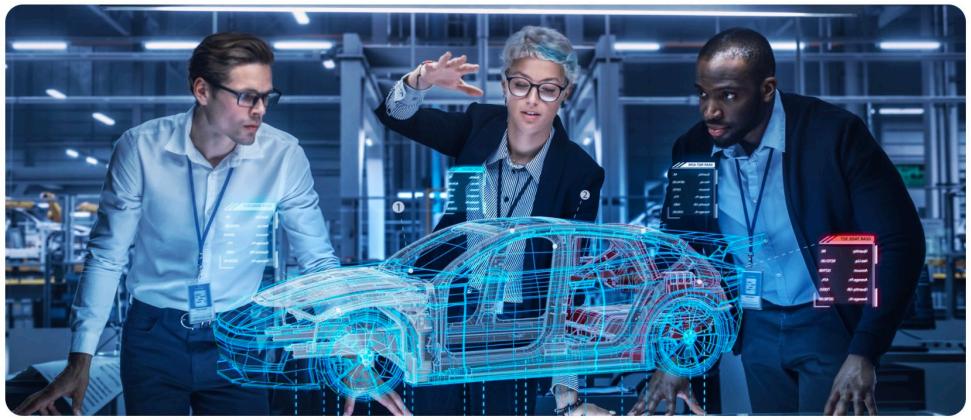


source: https://www.nissan-global.com/EN/INNOVATION/TECHNOLOGY/ARCHIVE/IAVM/

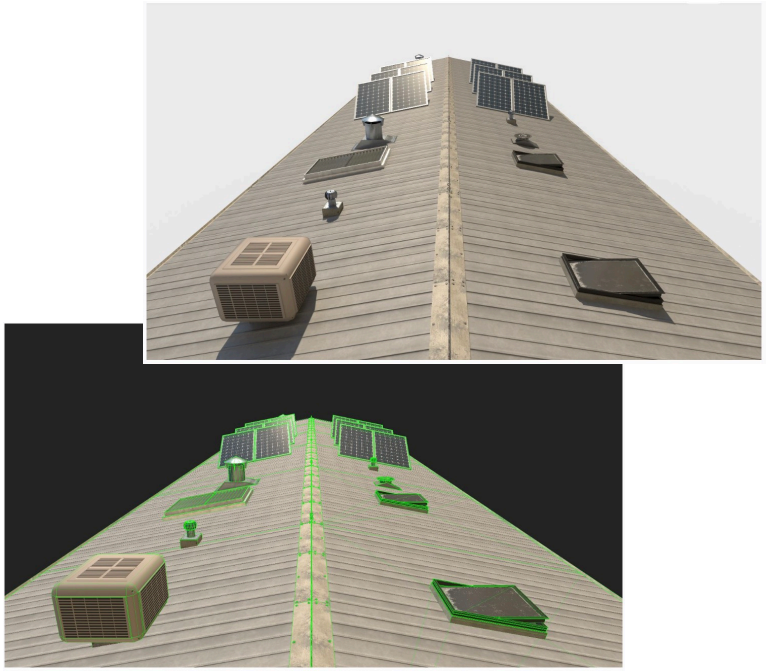
Applications of Low-poly Models

Module III: 3D Computer Graphics

- Low-poly models for VR and AR in the automotive industry
- Low-poly models in the manufacturing



source: https://vection-technologies.com/solutions/industries/automotive/



source: https://free3d.com/3d-model/industrial-roof-elements-3d-model-ar-vr-gameready-9890.html?dd_referrer=https://www.google.com/

Applications of High-poly Models

Module III: 3D Computer Graphics

- High-poly objects facilitate photorealistic representations, and users can zoom in to take a deeper look at the features. Here are the key uses:
  - You can use high-poly rendering to showcase any object that can be segregated into cross-sections.
  - You can deploy high-poly 3D rendering to develop renders of homes, engineering materials.
  - If you need to showcase your products using images of the highest definition, then high-poly modeling should be your choice.
  - You can use high-poly models to engage your target audience and showcase the detailing of your product.

Applications of High-poly Models

Module III: 3D Computer Graphics

- High-poly objects facilitate photorealistic representations, and users can zoom in to take a deeper look at the features. Here are the key uses:
  - High poly models should be your option if you are looking for visual richness and accuracy, and you are less bothered about interactivity.
  - High-poly models are great for creating static images with limited animations.
  - If you need to model a limited batch of assets and create high-quality outcomes, you may opt for high-poly modeling.

Applications of High-poly

Module III: 3D Computer Graphics

- Owing to high levels of detailing, high-poly 3D models are mostly used for depicting intricate details like:
  - design elements,
  - lifestyle images,
  - silo renders,
  - animations with zoom effects,
  - characters in games

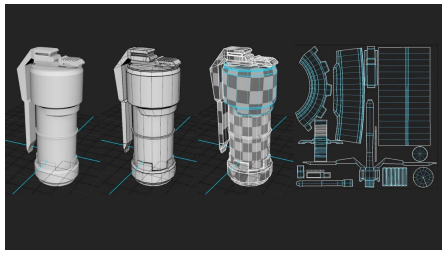


source: https://archicgi.com/case-studies/office-3d-rendering-5-home-offices/





source: https://retrostylegames.com/blog/game-studios-that-use-blender/



source: https://it-s.com/how-to-design-a-3d-model-using-3d-maya/



source: https://cgifurniture.com/blog/3d-rendering-guide/silo-product-image/



source: https://www.autodesk.com/products/maya/overview

silo rendering (n) - is a type of image in which a product is depicted against a seamless white or neutral background. The main purpose of it is to show a product in the cleanest and simplest way possible.

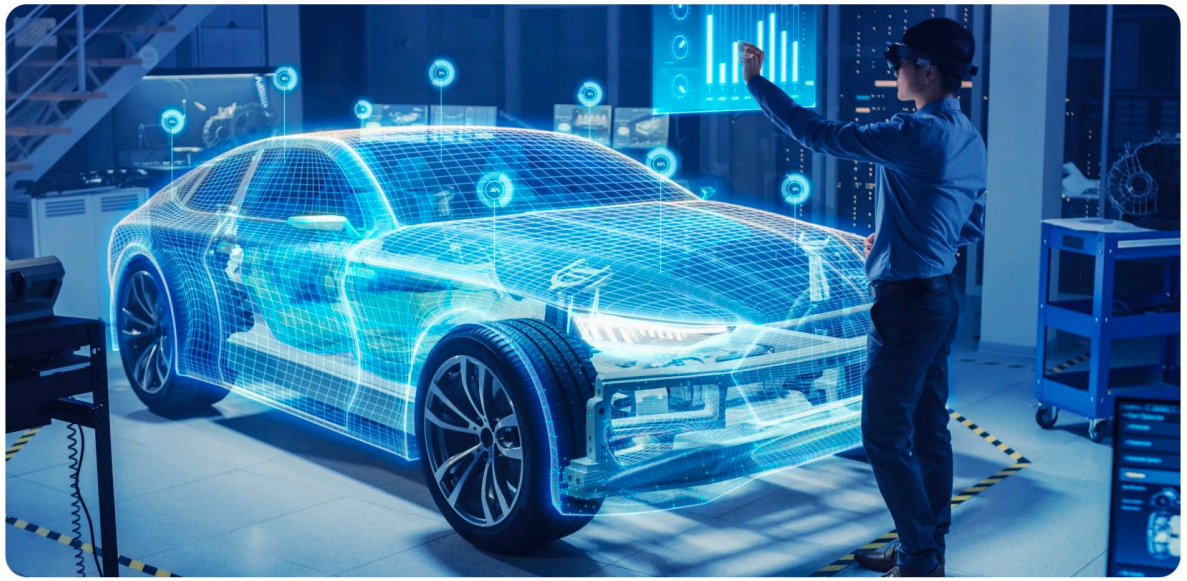
Applications of High-poly

Module III: 3D Computer Graphics

- High-poly model for VR and AR in the automotive industry
- High-poly model for VR and AR in the manufacturing



source: https://vection-technologies.com/solutions/industries/machinery-manufacturing/



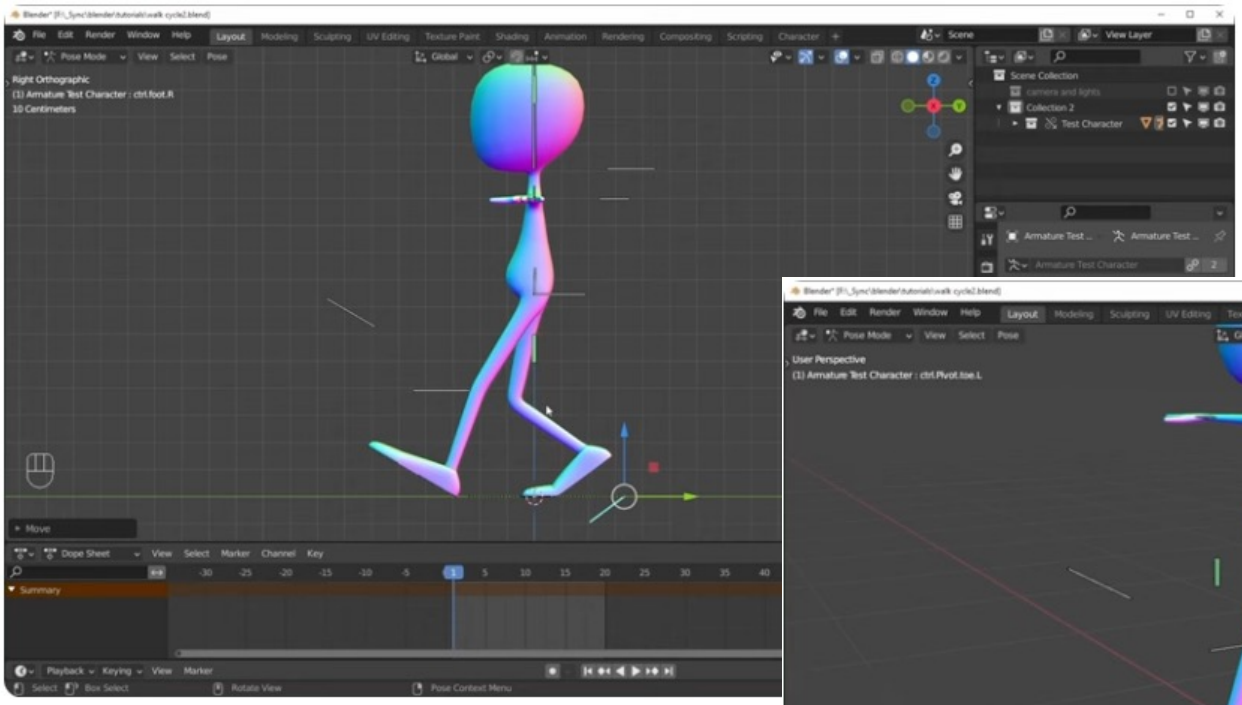
source: https://vection-technologies.com/solutions/industries/automotive/

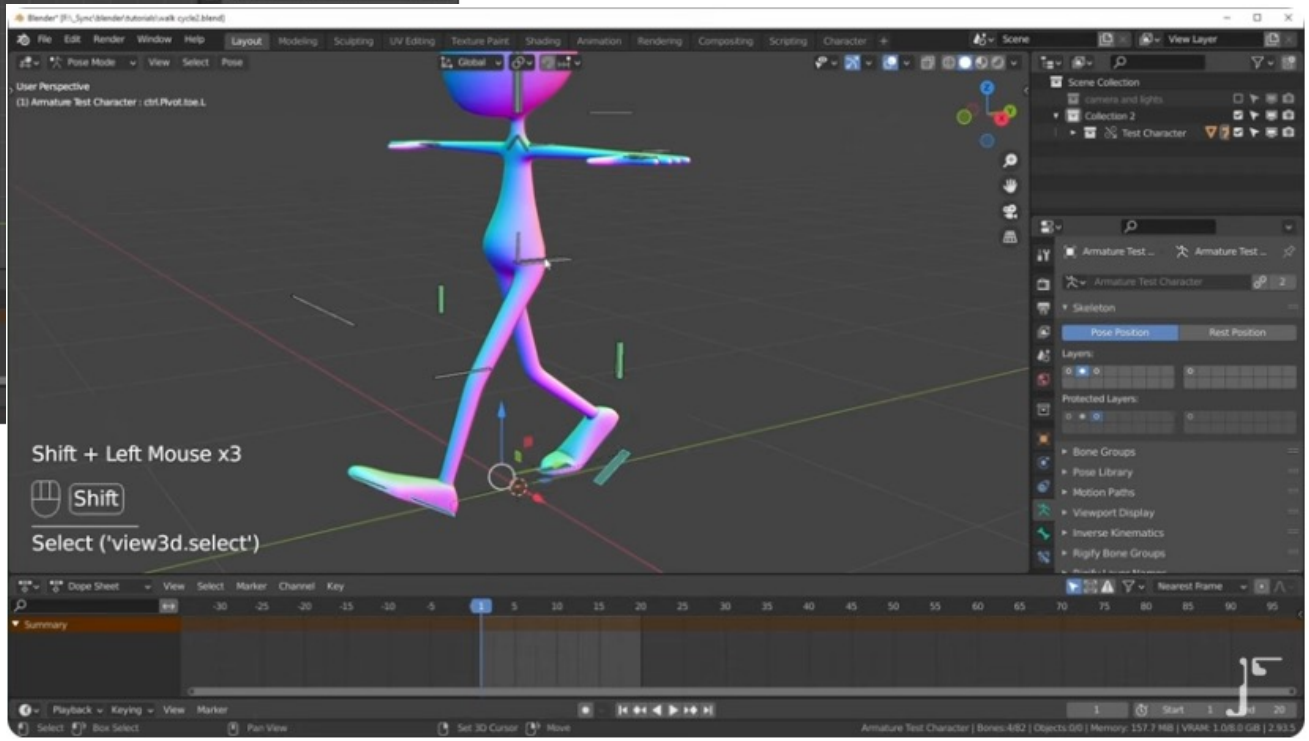
Animation Concept (3D Animation) cont’

Animation Techniques

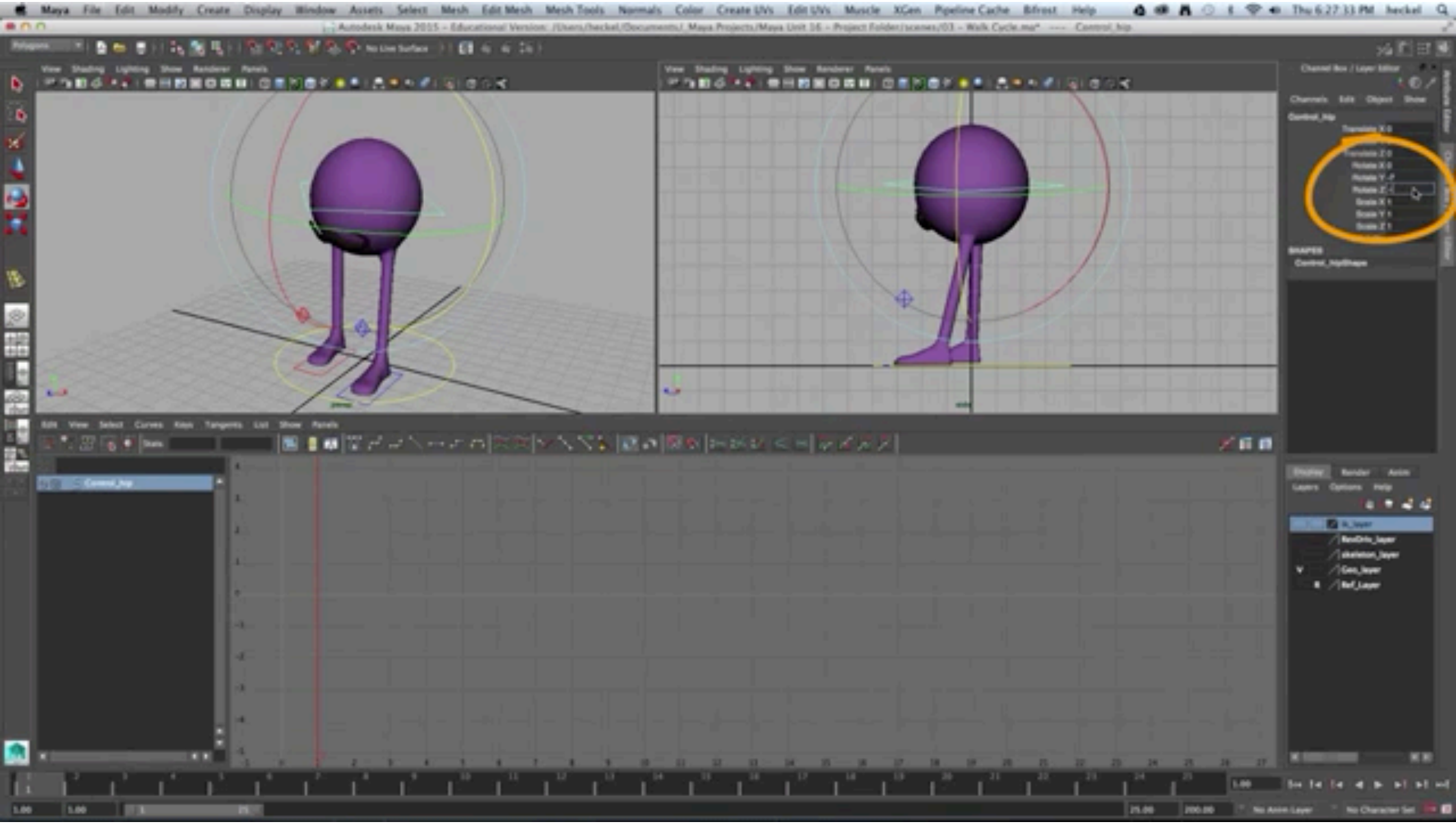
Module III: Animation/Video

- Animation techniques will be discussed:
  3. 3D Animation
     - 3D Animation, or Computer Generated Imagery (CGI), has become the dominant format in contemporary animated films.
     - It operates in a three-dimensional space, using digital models of characters, objects, and environments.
     - The process involves creating detailed 3D models and using advanced software to generate complex movements and interactions.
     - 3D animation offers versatility, capable of rendering hyper-realistic visuals or stylized artistic interpretations





source: https://www.youtube.com/watch?v=nRtT7Gr6S2o


Maya File Edit Modify Create Display Window Assets Select Mesh Edit Mesh Mesh Tools Normals Color Create UVs Edit UVs Muscle XGen Pipeline Cache Bifrost Help
Thu 6:27:11 PM heckel
Autodesk Maya 2015 – Educational Version: /Users/heckel/Documents/_Maya Projects/Maya Unit 16 – Project Folder/scenes/01 – Walk Cycle.ma*
Control_hip
View Shading Lighting Show Renderer Panels
Channel Box / Layer Editor
Channels Edit Object Show
Translate X 0
Translate Y 0
Translate Z 0
Rotate X 0
Rotate Y -7
Rotate Z 0
Scale X 1
Scale Y 1
Scale Z 1
SHAPES
Control_hipShape
Edit View Select Curves Keys Tangents List Show Panels
Display Render Anim
Layers Options Help
RL_layer
BendOrtu_layer
skeleton_layer
Geo_layer
Ref_Layer

Animation Techniques

Module III: Animation/Video

- Animation techniques will be discussed:
  4. Motion Capture
     - Motion capture, sometimes referred to as mocap or mo-cap, for short, is the process of recording high-resolution movement of objects or people into a computer system.
     - It is used in military, entertainment, sports, medical applications, and for the validation of computer vision and robots.





Motion Analysis

Simple

Step-by-Step

Create Motion Analysis Function

Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Add the `analyze_motion( )` function.

```
def analyze_motion(flow, frame_shape, grid_size):
    h, w = frame_shape[:2]
    y, x = np.mgrid[grid_size//2:h:grid_size, grid_size//2:w:grid_size].reshape(2,-1).astype(int)

    # recording all motion vector values in terms of magnitude and direction
    all_dx = []
    all_dy = []
    magnitudes = []
    directions = []
```

Create Motion Analysis Function

Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Add the `analyze_motion( )` function.

```
    for xi, yi in zip(x, y):
        dx = flow[yi, xi, 0]
        dy = flow[yi, xi, 1]

        # magnitude and direction conversion
        magnitude = np.sqrt(dx**2 + dy**2)
        direction = np.degrees(np.arctan2(dy, dx)) % 360  # convert to 0-360 degrees

        all_dx.append(dx)
        all_dy.append(dy)
        magnitudes.append(magnitude)
        directions.append(direction)
```

Create Motion Analysis Function

Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Add the `analyze_motion( )` function.

```
    return {
        'dx': np.array(all_dx),
        'dy': np.array(all_dy),
        'magnitudes': np.array(magnitudes),
        'directions': np.array(directions)
    }
```

Create Motion Analysis Function

Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
while frame_count < 100:
    ret, frame = cap.read()
    if not ret:
        print(f"End of video or error after {frame_count} frames.")
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None, **fb_params)

    # grid point for demonstration
    h, w = gray.shape[:2]
    y, x = np.mgrid[grid_size//2:h:grid_size, grid_size//2:w:grid_size].reshape(2,-1).astype(int)

    # analysis
    motion_data = analyze_motion(flow, gray.shape, grid_size)
```

Create Motion Analysis Function

Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
    # Calculate statistics for features of interest
    avg_magnitude = np.mean(motion_data['magnitudes'])
    avg_direction = np.degrees(np.arctan2(
        np.mean(motion_data['dy']),
        np.mean(motion_data['dx'])
    )) % 360

    # display
    vis_frame = frame.copy()
```

Create Motion Analysis Function

Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
    # small vectors (details)
    for xi, yi in zip(x, y):
        dx = flow[yi, xi, 0]
        dy = flow[yi, xi, 1]
        cv2.arrowedLine(vis_frame, (xi, yi),
                        (int(xi + dx*vector_scale), int(yi + dy*vector_scale)),
                        vector_color, 1, tipLength=0.3)

    # analysis display (2 frames calculation)
    cv2.putText(vis_frame, f"Avg Speed: {avg_magnitude:.2f} pixels/frame",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
    cv2.putText(vis_frame, f"Avg Direction: {avg_direction:.1f} degrees",
                (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
```

Create Motion Analysis Function

Module III: Simple Analysis

- Improve the Farneback code by adding a section for analyzing the direction and speed of movement:
  - Improved the code content in the motion calculation loop.

```
    # overall summary (2 frames analysis)
    center_x, center_y = w // 2, h // 2
    avg_dx = np.mean(motion_data['dx'])
    avg_dy = np.mean(motion_data['dy'])
    cv2.arrowedLine(vis_frame, (center_x, center_y),
                    (int(center_x + avg_dx*vector_scale*3),
                     int(center_y + avg_dy*vector_scale*3)),
                    (0, 0, 255), 3, tipLength=0.5)

    # update Widget
    image_widget.value = cv2.imencode('.png', vis_frame)[1].tobytes()

    prev_gray = gray.copy()
    frame_count += 1
```

Direction

Module III: Simple Analysis

- Mean Direction (Degrees): Calculated from arctan2(`dy`, `dx`) and converted to degrees.
- Direction Interpretation:
  - 0°: Movement to the right
  - 90°: Movement downward
  - 180°: Movement to the left
  - 270°: Movement upward

Magnitude

Module III: Simple Analysis

- Vector size (pixels/frame): Calculated from sqrt(`dx^2 + dy^2`)
- Relative speed:
  - < 0.5: Very slow
  - 0.5-2: Slow
  - 2-5: Moderate
  - 5: Fast

Motion Analysis

Module III: Simple Analysis



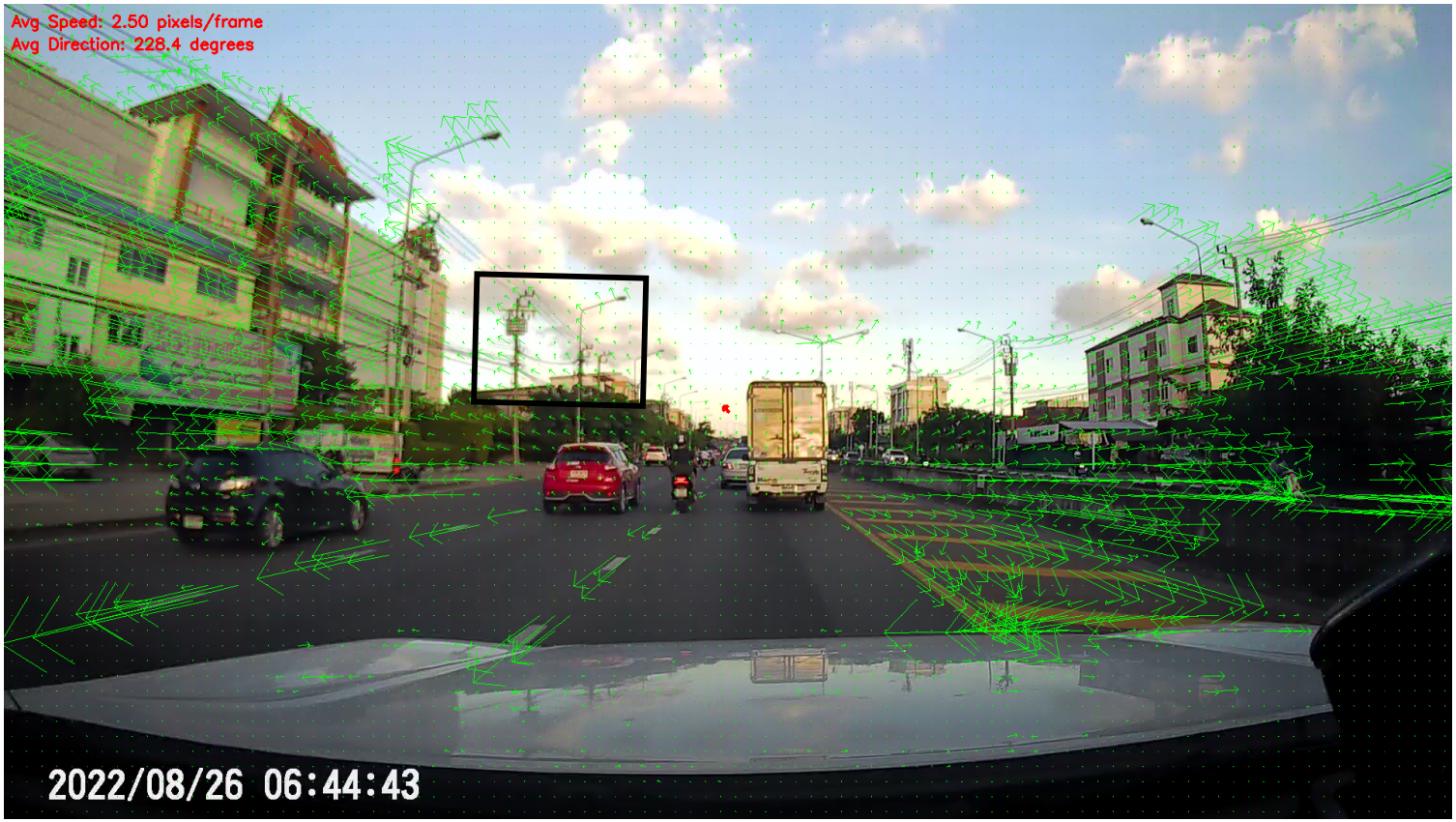
Practical Application

Module III: Motion Vector Field

- Primary Motion Detection:
  - Checks which direction the object is moving.
  - Checks for significant movement.
- Abnormality Detection:
  - Checks when the average speed exceeds a specified threshold.
  - Checks when the direction of movement deviates from the normal pattern.
- Flow Analysis:
  - Analyze fluid movement.
  - Study the main flow directions.

Motion Analysis

Module III: Speed & Direction



Motion Analysis

Module III: Fluid/Gas/Smoke Movement



Voice AI

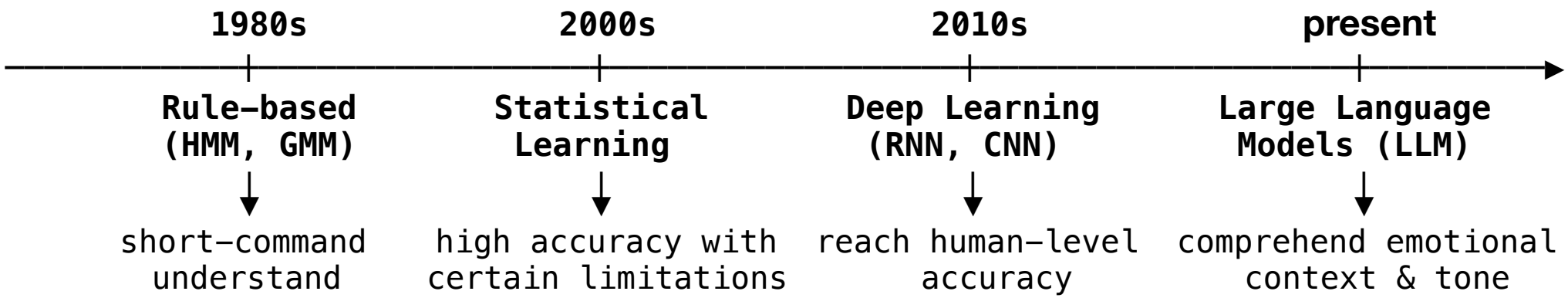
Voice AI

- Voice AI, or artificial intelligence in speech, is technology that enables machines to listen, understand, respond, and synthesize human speech, just as we converse with people in general.

Humans do	Voice AI does
Listen to sound	Receive sound via microphone
Distinguish/recognize speech	Convert speech to text (Speech-to-Text)
Understand the meaning	Process language (NLP)
Know what emotion the speaker has	Detect emotion from voice (Emotion Recognition)
Respond	Speak back with synthetic voice (Text-to-Speech)

Evolution of Voice AI

- Voice AI has been under development since humans began using computers.
- A simple timeline of Voice AI development can be explained as shown in the image.



Evolution of Voice AI in 1980s

- The 1980s were a pioneering era (Rule-based).
- The main technologies were Rule-based, HMM (Hidden Markov Model), and GMM (Gaussian Mixture Model).
- Working principles:
  - Uses basic linguistic and statistical rules.
  - All rules must be written manually.
  - Only the taught terminology is applicable.

Evolution of Voice AI in 1980s

- Real-life examples:
  - Voice-operated calculators
  - Older in-car voice command systems
  - Automated phone menus that say "Press 1 for..."
- Limitations:

```
# Assume it has only been trained on 10 words
Trained words = ["open", "close", "light", "TV", "music"]

# If you say "open the TV light please" → it gets confused immediately! 😕
# Because the word "please" isn't in its vocabulary
```

  - Only know a few words like 'mom', 'dad', and 'rice'; like a child who can't form full sentences yet.

Evolution of Voice AI in 2000s

- The 2000s were the era that introduced Statistical Learning into speech technology.
- Core technologies: GMM-HMM, SVM, and other Statistical Models.
- How it worked:
  - Used statistical models to aid in decision-making.
  - Began collecting user data to refine and improve system performance.
  - Became capable of recognizing a much wider vocabulary.

Evolution of Voice AI in 2000s

- Real-life examples:
  - Bank phone systems: "Say the name of the menu you want."
  - Voice-activated GPS navigation systems.
  - Dictation software (speech-to-text).
- Advantages:

```
# Starts to understand similar/related words
Previously encountered = ["turn on the light", "turn off the light"]

# If you say "turn on the big light"
# → It can guess/interpret it as "turn on the light" + "big"
```

  - Like a child who is starting to learn a language—they know more vocabulary now, but they still get confused by idioms or phrases that aren't an exact, word-for-word match.

Evolution of Voice AI in 2010s

- The 2010s was the era of the Deep Learning revolution.
- Core technologies: Deep Learning, RNN, LSTM, CNN, Transformer.
- How it worked:
  - Learned features and patterns automatically from data (no need to manually program rules).
  - Used multi-layered artificial neural networks.
  - Required massive amounts of data (Big Data).

Evolution of Voice AI in 2010s

- Real-life examples:
  - Siri/Google Assistant – can engage in two-way, conversational interactions.
  - Alexa/Google Home – smart speakers.
  - Google Live Transcribe – provides real-time speech-to-text transcription.
  - YouTube Auto-Caption – automatically generates subtitles/ captions for videos.

Evolution of Voice AI in 2010s

- Increased Capabilities:

```
# Before the Deep Learning era
user: "Will it rain tomorrow?"
AI: (Must contain the exact words "weather forecast" to trigger a response)

# Deep Learning era
user: "Will it rain tomorrow?", "How's the weather tomorrow?", "Are long
pants a good idea?"
AI: Understands that all of these questions are related to the weather (or
are interpreted within the weather context)
```

  - It's like a multilingual person—they grasp the context, so even if you don't speak perfectly or literally, they still understand what you mean.

Evolution of Voice AI Today

- The present is the era of Large Language Models (LLMs).
- AI now understands context, engages emotionally, and interacts naturally.
- Core technologies: Transformer, GPT, BERT, Multimodal AI.
- How it works:
  - Learns from massive datasets (the entire internet).
  - Understands context, emotion, and tone.
  - Responds in a human-like manner.

Evolution of Voice AI Today

- Real-life examples:
  - ChatGPT Voice – engages in natural, conversational dialogue.
  - Google Gemini – understands images, audio, and text together (multimodal).
  - AI Call Center – analyzes customer emotions and sentiment in real time.
  - AI Therapist – provides mental health counseling and support.

Evolution of Voice AI Today

- Capabilities:

```
# Natural conversation
user: "I'm so tired today..."
AI: "What happened? 🥺 You sound really discouraged."
    (Detects tone and chooses an empathetic response.)

# Emotion understanding
user: "I'm extremely dissatisfied!"
AI: Detects angry tone → responds politely and offers a solution.
```

Evolution of Voice AI Today

- Key Advantages:
  - Remembers conversation context (knows what has been discussed earlier).
  - Adapts tone and delivery style to match the interlocutor.
  - Expresses emotion through synthesized voice (Emotional TTS).
  - Works seamlessly with multimodal data (image, audio, and text).

Emotion Recognition

- Voice AI, or Voice Artificial Intelligence, is technology that enables machines to listen, understand, respond, and synthesize human speech—just like we do when talking to other people.

Spoken Phrase	Voice Characteristics	Detected Emotion
"Yay! That's awesome!"	High pitch, energetic	Happy 😊
"What... why did this happen?"	Low pitch, slow	Sad 😢
"Come on! That's enough!"	Loud, harsh	Angry 😠
"Okay... alright then."	Flat, neutral	Neutral 😐

Voice AI Yesterday

- Voice AI you may have already used:
  - Siri / Google Assistant / Alexa – voice-based Q&A and conversational interaction.
  - Google Voice Search – search using your voice.
  - Spotify Podcast Transcript – provides podcast transcriptions/captions.
  - Bank Call Center – "Press 1 or say..." (voice-enabled IVR menus).

Voice AI Tomorrow

- Trending Voice AI applications:
  - AI Call Center – analyzes customer emotions during live conversations.
  - AI Therapist – detects emotional states in patients with depression.
  - AI in Gaming – adapts the storyline based on the player's emotions.
  - AI Voice Cloning – replicates the voices of celebrities/famous people.

Voice AI applications

- Trending Voice AI applications:

Domain	Benefits
Business	24/7 customer service, satisfaction analysis
Education	Personal teaching assistants, language practice
Healthcare	Mental health patient monitoring, assistance for people with disabilities
Entertainment	Emotion-adaptive games, audiobooks
Daily Life	Voice commands, smart home control

The Technology Behind Voice AI

- Voice AI is technology that enables machines to listen, understand, respond, and synthesize human speech—just like we do when talking to other people.

```
Speech (Audio Input)
        ↓
[ Preprocessing ]  ← Signal conditioning, noise reduction
        ↓
[ Feature Extraction ] ← Extract features: MFCC, Pitch, Energy
        ↓
[ Machine/Deep Learning Model ] ← CNN, LSTM, Transformer
        ↓
[ Output ] ← Text / Emotion / Speaker Identity
```

Voice AI Development Tools

Tools for Voice AI

- Step 1: Prepare the environment (run once).

```
# @title Install the necessary libraries.
!pip install librosa soundfile matplotlib numpy scikit-learn -q

import librosa
import soundfile as sf
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Audio, display
import warnings
warnings.filterwarnings('ignore')

print("Now available!")
```

```
Now available!
```

Tools for Voice AI

- Line 1: Load all the tools we will use to create Voice AI.

```
!pip install librosa soundfile matplotlib numpy scikit-learn -q
```

- -q = quiet mode (hides text during installation.)

Tools for Voice AI

- `It's like building a house; you need the tools first. This line loads all the tools we'll be using today.`

Tool	Purpose in Voice AI
librosa	Audio analysis and feature extraction (e.g., MFCC, chroma, tempo, pitch). Essential for preparing audio data for ML models.
soundfile	Handles reading and writing audio files in formats like WAV, FLAC, and OGG.
matplotlib	Visualizes audio signals, spectrograms, and waveform plots for analysis and debugging.
numpy	Provides numerical array operations that power most audio processing and ML computations.
scikit-learn	Offers traditional ML algorithms (SVM, GMM, etc.) for classification tasks like speaker or emotion recognition.

Tools for Voice AI

- Line 2: import librosa

```
import librosa
```

- When we want to use a camera, we first have to take it out of its case; import is like taking a tool out of the box to use it.
- What do we use it for?
  - Load audio files (.wav, .mp3)
  - Extract audio features such as MFCC
  - Analyze frequency and energy
  - Display spectrograms
- Example librosa.load('audio.wav')

```
librosa.load('audio.wav')  # ← Reads the audio file
```

Tools for Voice AI

- Line 3: import soundfile as sf

```
import soundfile as sf
```

- librosa can read files, but soundfile allows us to save newly created audio files.
- We use it to:
  - Save audio that we have generated/created ourselves.
  - Convert audio files between different formats.
  - Check/inspect audio quality and file properties.
- Example sf.write('newaudio.wav', audio, 16000)

```
sf.write('newaudio.wav', audio, 16000)  # ← Saves the audio file
```

Tools for Voice AI

- `Line 3: import soundfile as sf`

```
import soundfile as sf
```

Code Example	What It Does
`sf.read('sound.wav')`	Reads an audio file and returns the data + sample rate
`sf.write('new.wav', audio, 16000)`	Saves audio data as a WAV file with a sample rate of 16,000 Hz
`sf.write('new.flac', audio, sr)`	Saves audio in FLAC format (lossless compression)
`sf.write('new.ogg', audio, sr)`	Saves audio in OGG format (compressed)

Key formats supported: WAV, FLAC, OGG, AIFF, and more.

Tools for Voice AI

- Line 4: import numpy

```
import numpy as np
```

- Sound is actually numbers. NumPy helps us manage these numbers easily.
- We use it to:
  - Generate sound waves (e.g., 0.5, -0.3, …).
  - Calculate averages, maximum values, etc.
  - Manage data as arrays.
- Example

```
# Sound is these numbers
sound = [0.1, 0.3, 0.5, 0.3, 0.1, -0.1, -0.3, -0.5, -0.3, -0.1]
np.mean(sound)  # Finds the average value
```

Tools for Voice AI

- Line 5: import matplotlib

```
import matplotlib.pyplot as plt
```

- Numbers alone are hard to understand. Matplotlib helps us draw pictures to visualize the data.
- We use it to:
  - Plot audio waveform graphs
  - Draw spectrograms
  - Compare different sounds
- Example

```
plt.plot(time, amplitude)  # Plots the graph
plt.show()  # Displays the image
```

Tools for Voice AI

- Line 6: import IPython.display

```
from IPython.display import Audio, display
```

- We also need to hear the sound. This is the audio player in Colab.
- We use it to:
  - Play audio directly inside the notebook.
  - Display an interactive audio player widget.
  - Listen to the sounds we have created.
- Example:

```
from IPython.display import Audio, display
...
display(Audio('sound.wav'))  # Shows a play button for the audio
```

Tools for Voice AI

- Line 7: import warnings

```
import warnings
warnings.filterwarnings('ignore')
```

- Sometimes the program shows warnings about unimportant things, making us focus on the wrong place. This line tells the program: "Don't warn us."
- Why ignore warnings?
  - Keeps the program output clean and uncluttered.
  - Helps us focus on the actual important content/results.
  - Prevents students/learners from getting confused by irrelevant messages.

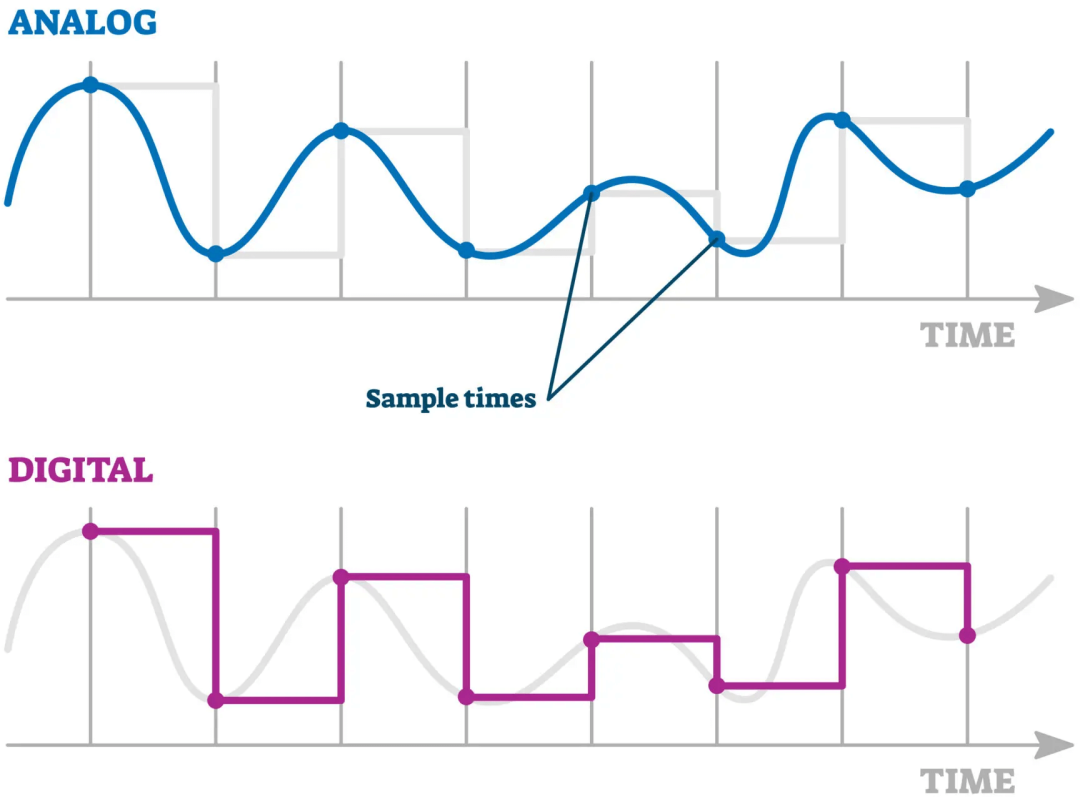
Sound Wave Component

Sound Wave: Waveform

- Sound is caused by the movement of molecules in the air.
- Sound travels through the air as waves—much like when we throw a stone into water and ripples spread outward.
- The ears of living beings, such as humans, perceive sound as analog signals.
- But computers perceive sound as digital signals.

Human Ear (Analog)	**Computer (Digital)**
Receives continuous sound waves	Samples sound at discrete time intervals
Like a smooth, unbroken curve	Like a series of dots/numbers approximating that curve
Responds to infinite variations in pitch and loudness	Represents sound as binary data (0s and 1s)

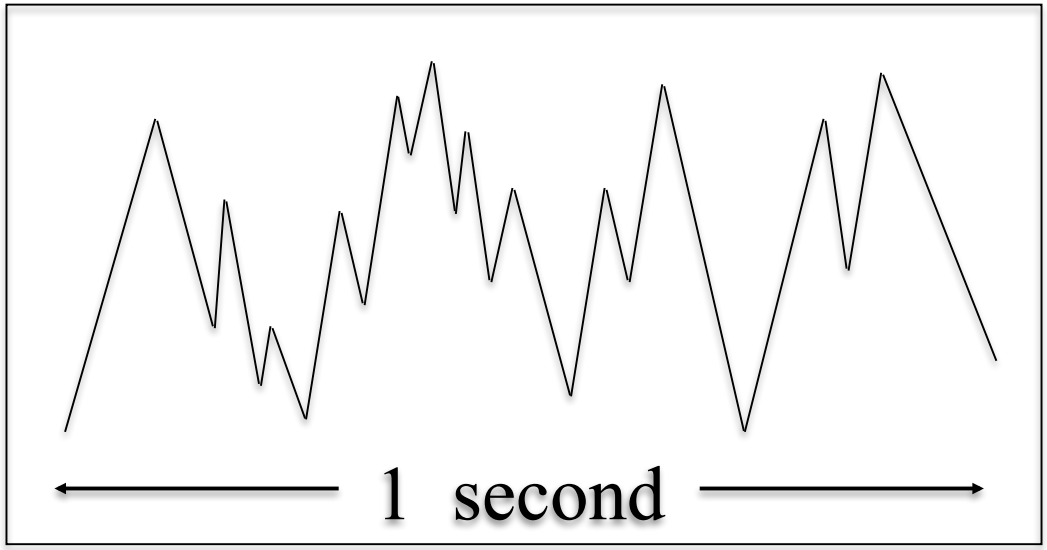
Simple Visualization



Reference: https://www.allelcoelec.com/blog/analog-vs.digital-signals.html?srsltid=AfmBOoopUaGinpOx6JjMg5JPEkP16iIjT2uU1KFLQ1eCEGAu_PM0CPMC

Sampling Rate

- Sampling Rate is the number of samples taken from an analog signal per second from a sound wave.

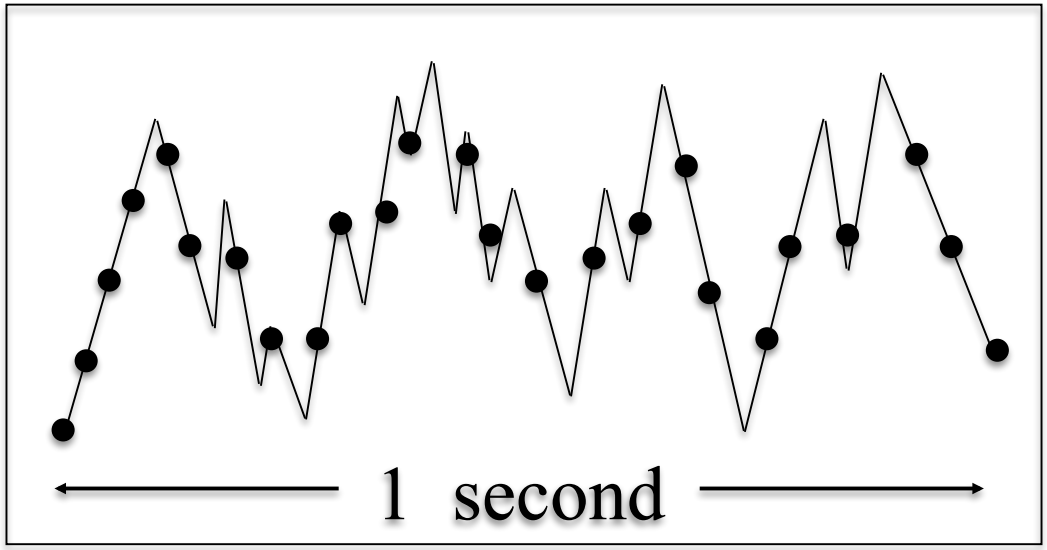


Continuous *(analog)*

Reference: Anthony J. Nowakowski presentation.

Sampling Rate

- The number of samples collected for each waveform is recorded digitally so the computer can use the audio data.

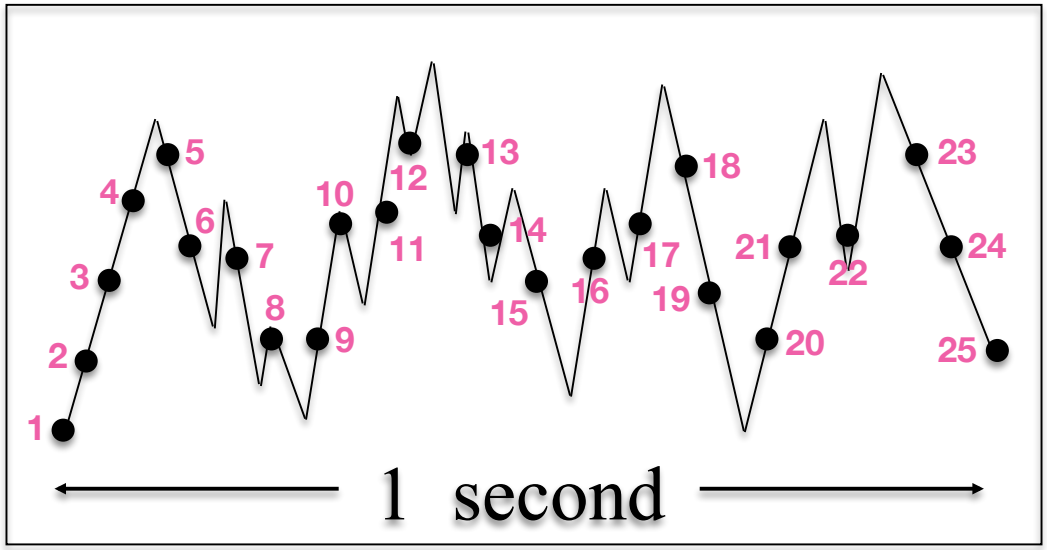


Sampled 25/sec. ***(digital)***

Reference: Anthony J. Nowakowski presentation.

Sampling Rate

- Sampling rate is the number of samples collected from an analog signal per second to record audio data that a computer can understand.

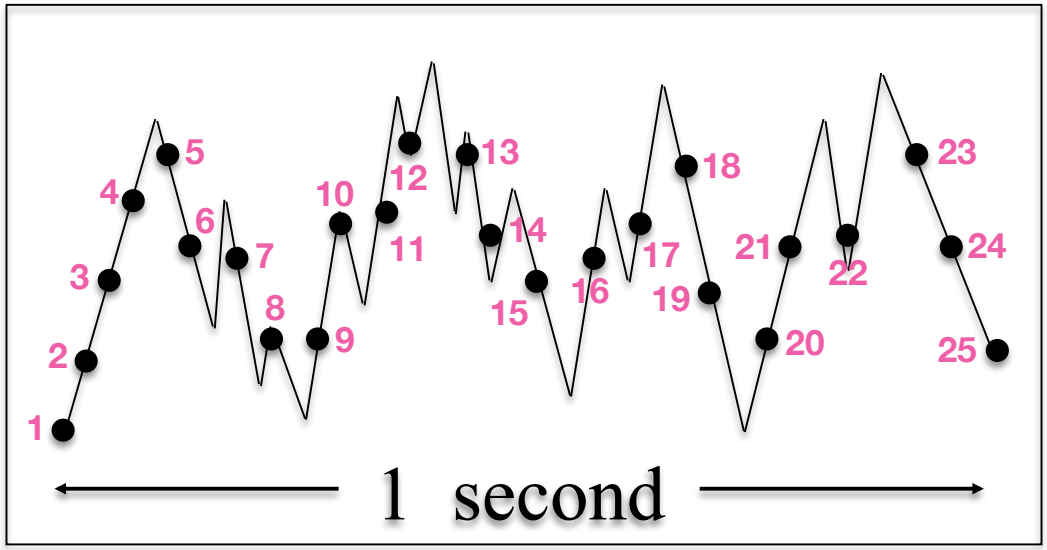


Sampled 25/sec. ***(digital)***

Reference: Anthony J. Nowakowski presentation.

Sampling Rate

- The number of times we sample a sound wave per second is measured in Hertz (Hz), which is the number of times the ADC (analog-to-digital converter) measures the frequency of the sound wave.

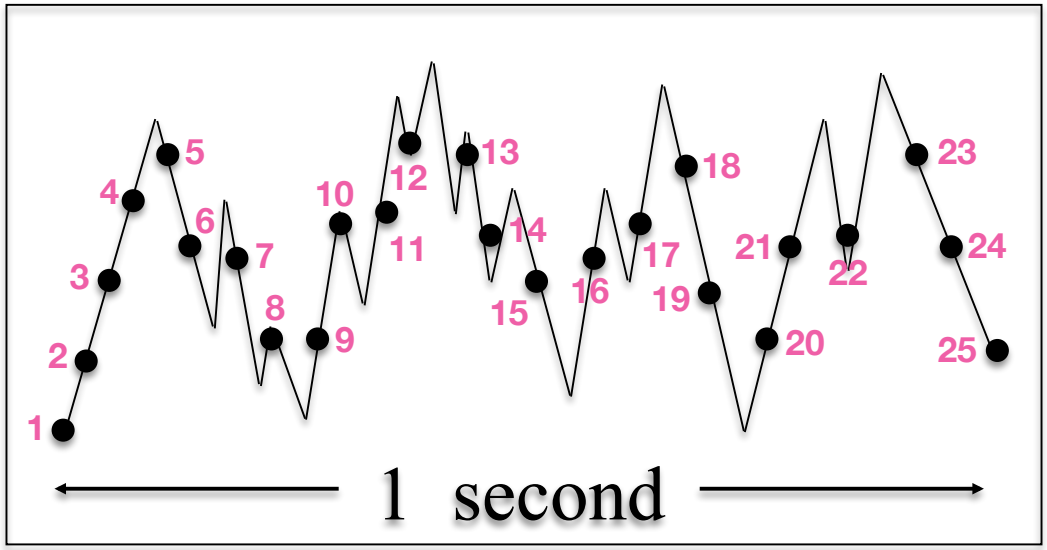


Sampled 25/sec. ***(digital)***

Reference: Anthony J. Nowakowski presentation.

Sampling Rate

- We will use digital signals to plot sound waves on a computer.



Sampled 25/sec. ***(digital)***

Reference: Anthony J. Nowakowski presentation.

Sampling Rate

- Simple Analogy
  - Think of it like frames per second (FPS) in a video:

Video	**Audio**
24 FPS = 24 pictures per second	16,000 Hz = 16,000 samples per second
More FPS = smoother motion	Higher sampling rate = better audio quality
Less FPS = choppy/jerky motion	Lower sampling rate = more muffled/lossy audio

Sampling Rate

- Common Sampling Rates

Sampling Rate	Quality Level	Used For
8,000 Hz (8 kHz)	Telephone quality	Voice calls, old landlines
16,000 Hz (16 kHz)	Good speech quality	Speech recognition (e.g., Whisper, Google Speech-to-Text), Voice AI
44,100 Hz (44.1 kHz)	CD quality	Music, high-quality audio
48,000 Hz (48 kHz)	Studio quality	Professional audio, film production

- Remember this line from earlier?

```
sf.write('audio.wav', audio, 16000)  # ← 16000 is the Sampling Rate!
```

Sampling Rate

- Does Sampling Rate Matter?

If Sampling Rate is...	Result
Too low (e.g., 4,000 Hz)	High-frequency sounds (like 's' and 'sh' sounds) are lost. Speech sounds muffled or unclear.
Just right (e.g., 16,000 Hz)	Captures all human speech frequencies clearly (human voice ranges from ~80 Hz to 8,000 Hz).
Very high (e.g., 48,000 Hz)	Captures very high frequencies (music, instruments) but uses more storage and processing power.

Nyquist Rule

- Nyquist Rule (Simple Version)
  - To properly capture a sound frequency, you must sample at “at least double” that frequency.
- For example:
  - Human speech goes up to about 8,000 Hz.
  - So we need at least 2 × 8,000 = 16,000 Hz sampling rate.
  - That's exactly why 16 kHz is the standard for Voice AI!

Sampling Rate

- Disadvantages of a High Sampling Rate
  - A high sampling rate results in larger audio files.
  - The more frequently we sample = the more data per second = the larger the file size.
- Important Note: The sampling rate is only one factor that determines file size.
  - Another factor is audio resolution (Bit Depth).

Visualizing the Trade-off

Sampling Rate	Data per Second	File Size (for 1 minute)	Quality
8,000 Hz	8,000 samples/sec	Very small	Telephone quality
16,000 Hz	16,000 samples/sec	Small	Good for speech (Voice AI standard)
44,100 Hz	44,100 samples/sec	~10 MB (CD quality)	High (music)
48,000 Hz	48,000 samples/sec	~11 MB	Studio quality

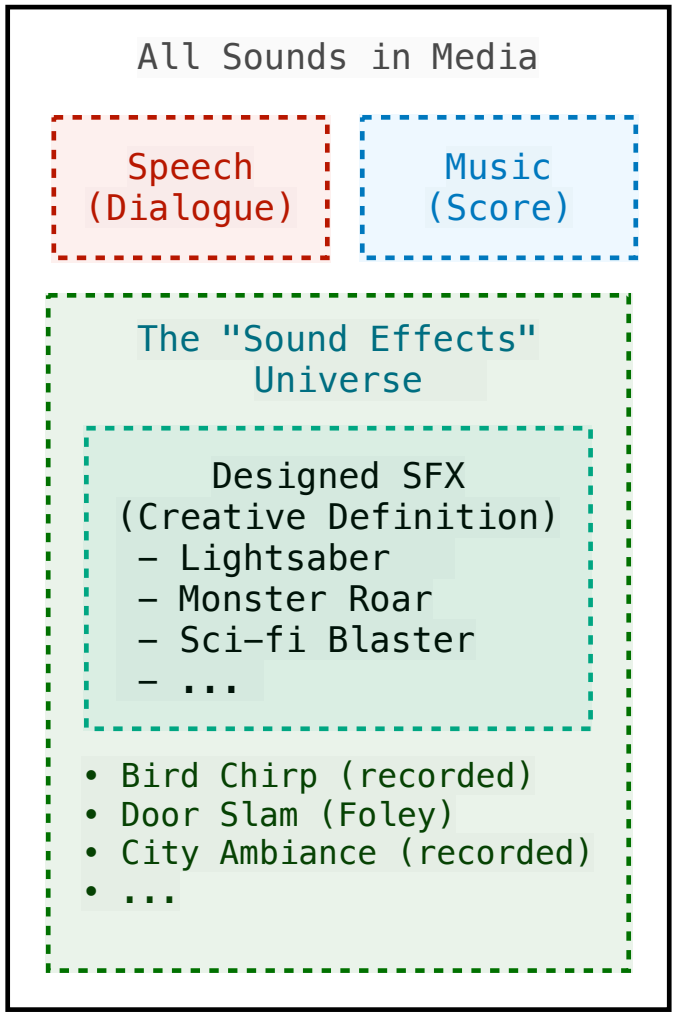
Note: File size also depends on Bit Depth and whether compression is used.

- Sampling Rate
  - How often we measure the sound wave (horizontal axis/time).
- Bit Depth
  - How precisely we measure each sample (vertical axis/amplitude).

Sound Effects

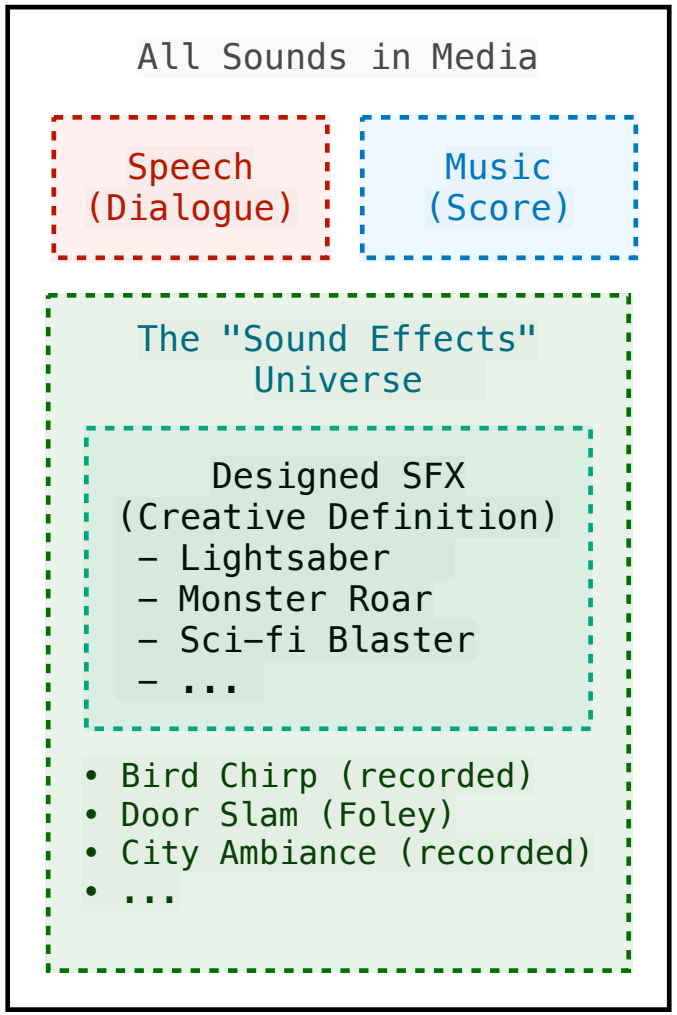
Sound Effects

- All sounds in media can be classified according to this diagram.
- It appears there are two ways to look at this—and both approaches are important:
  - Functional Definition
  - Creative Definition



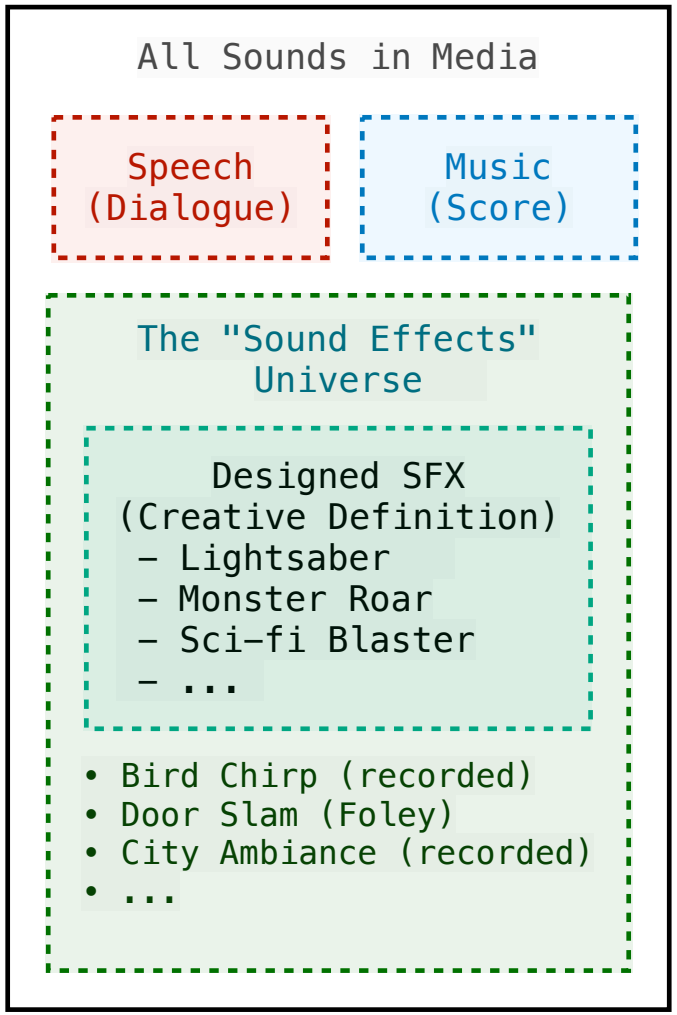
Sound Effects

Perspective	Functional Definition	Creative Definition
Focus	Purpose and utility	Artistry and emotion
What it cares about	Conveying clear information, alerts, navigation, dialogue comprehension	Setting mood, atmosphere, storytelling, emotional impact
Examples	Voice commands, warning alarms, clear speech, notification sounds, call center IVR	Film scores, background music, Foley sound effects, immersive game audio, emotional TTS
Voice AI Connection	Speech-to-Text accuracy, command recognition, clear TTS for instructions	Emotional TTS, expressive voice design, adapting tone to context



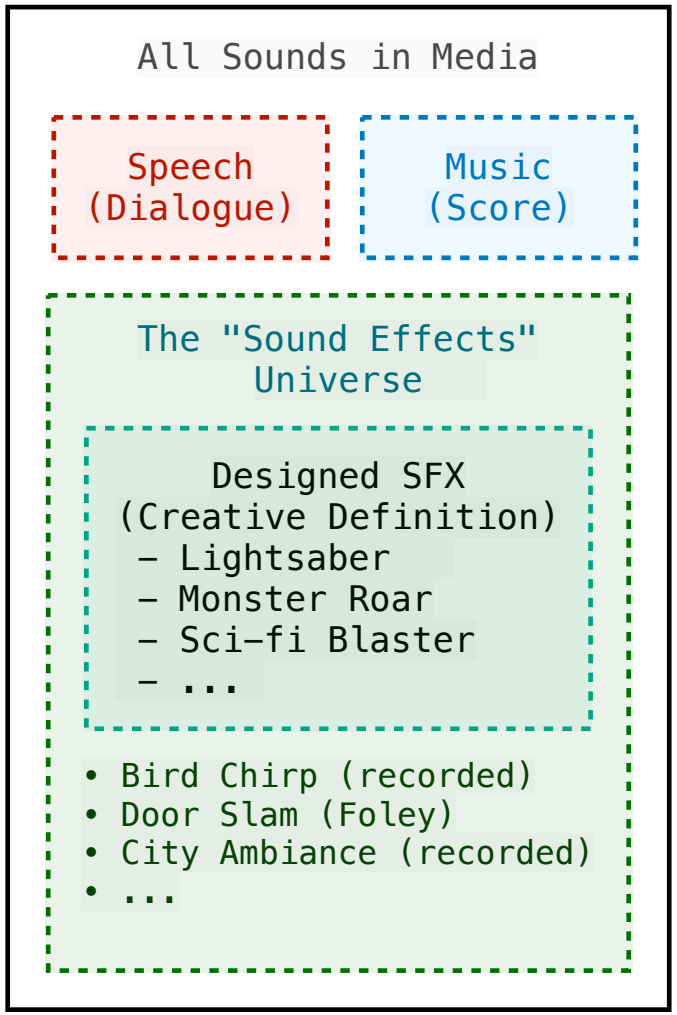
Sound Effects

- The functional definition covers this entire "universe of sound effects".
  - That is, everything that is not dialogue or music.
  - This includes both sounds recorded from the real world and entirely newly created sounds.
  - From natural sounds like birds chirping to completely artificial ones, such as lightsaber sounds.



Sound Effects

- The creative definition refers specifically to design within that universe.
  - It is a subset of sounds that have been modified, enhanced, or created from scratch.
  - A recorded bird chirp is a 'sound effect' under the functional definition.
  - But if we lower the pitch, slow it down, and layer it with an animal's roar to create a terrifying monster screech...
  - ...then we have entered the realm of the creative definition.

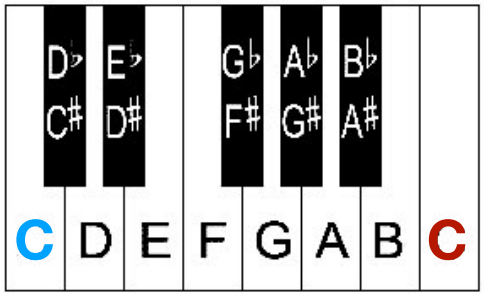


Creating Sound Waves

Pitch

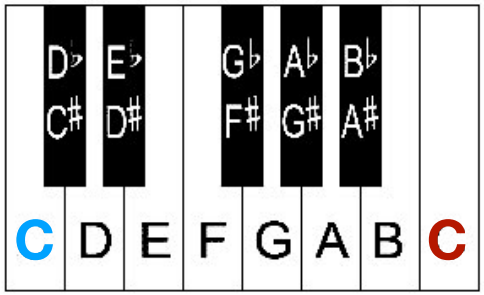
- Music note names are very short – using only a few letters from our English alphabet.
- They are the first seven letters: A, B, C, D, E, F, and G.
- They are arranged in order, just as we say them: A...B...C... and so on.
- The tricky part is what to do when we reach G? – we don't stop... we start over!

```
 C  D  E  F  G  A  B  C  D  E  F  G  A  B  C  D  E  F  G ...
|←     1 Octave     →|←    1 Octave     →|
```



Pitch

- Notes are arranged like this: A, B, C, D, E, F, G... A, B, C... and so on, until we reach the end of the piano keyboard.
- The 8th C note is called one octave above the first C note.
- It is also the first note of the next set of 7 notes – the next octave.
- The reason we use the same note names repeatedly is that all the first notes sound similar, but at a different pitch.



Waveform

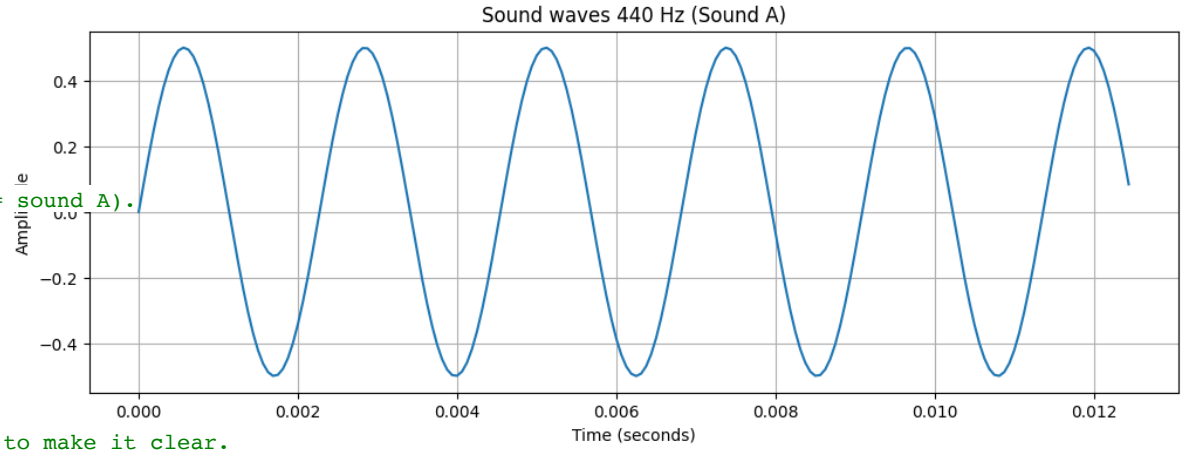
- Example: Create simple sounds to see what sound waves look like.

```
# @title Create simple sound waves.
import librosa
import soundfile as sf
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Audio

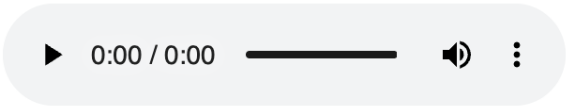
# Generate a sine wave (a sound with a frequency of 440 Hz = sound A).
sr = 16000  # sampling rate
duration = 0.5  # 0.5 seconds
t = np.linspace(0, duration, int(sr * duration))
freq = 440  # frequency
audio = 0.5 * np.sin(2 * np.pi * freq * t)

# Show graph
plt.figure(figsize=(12, 4))
plt.plot(t[:200], audio[:200])  # Just the first 200 points to make it clear.
plt.title("Sound waves 440 Hz (Sound A)")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

# Listen to the sound.
print("Listen to the sound:")
display(Audio(audio, rate=sr))
```



Listen to the sound:



Waveform

- Step 1. Parameters configuration

```
sr = 16000  # Sample rate: 16 kHz
duration = 0.5  # Audio length: 0.5 seconds
freq = 440  # Frequency: 440 Hz (A note/La)
```

  - sr (Sample Rate) = 16,000 samples/second – the number of data points used to capture audio in 1 second.
  - duration = 0.5 seconds – creates a short, brief sound.
  - freq = 440 Hz – the standard frequency for the musical note A (La).

Waveform

- Step 2. Creating the Time Axis

```
t = np.linspace(0, duration, int(sr * duration))
```

  - Creates a time range from 0 to 0.5 seconds.
  - Has a total of 8,000 points (16,000 × 0.5).

Waveform

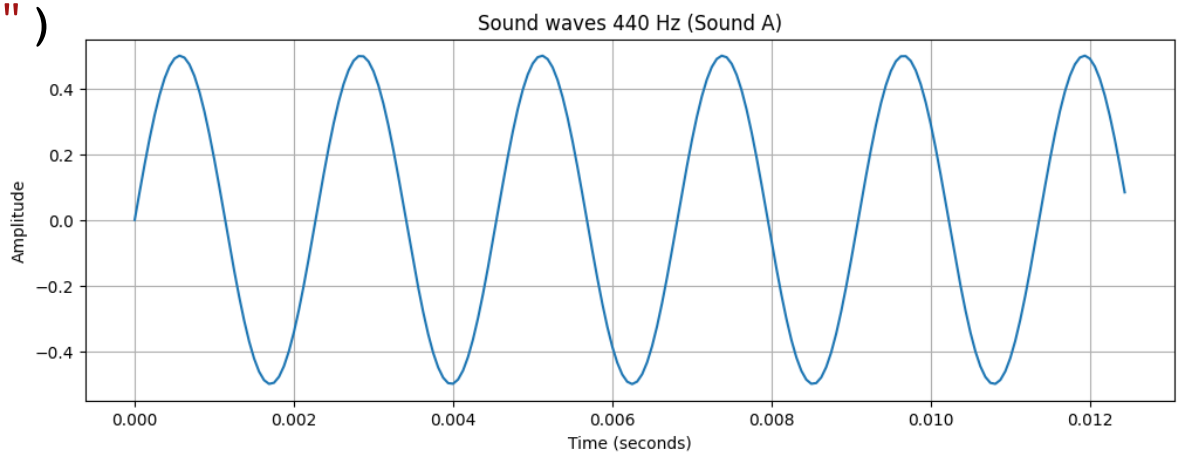
- Step 3. Create sound waves.

```
freq = 440  # frequency
audio = 0.5 * np.sin(2 * np.pi * freq * t)
```

Waveform

- Step 4. Displaying the waveform of the generated audio.

```
# Show graph
plt.figure(figsize=(12, 4))
plt.plot(t[:200], audio[:200])  # Just the first 200 points.
plt.title("Sound waves 440 Hz (Sound A)")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



- Graph: Shows the first 200 points of the wave.

Waveform

- Step 5. Listen to the sound.

```
# Listen to the sound
print("Listen to the sound:")
display(Audio(audio, rate=sr))
```

- We use display(Audio(audio, rate=sr)) to play that sound 🎧🎵

```
Listen to the sound:
```



References

Reference Materials

- John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, and Kurt Akeley, “Computer Graphics,” Third Edition, *Addison-Wesley*, 2014
- P. Joshi, D. M. Escriva and V. Godoy, “OpenCV by Example,” *Packt Publishing*, Birmingham - Mumbai, 297 pages, ISBN 978-1-78528-094-8, Jan 2016.
- MediaPipe: https://mediapipe.readthedocs.io/en/latest/solutions/hands.html, accessed August 18, 2025

Mahidol University